

A CONCEPTUAL GUIDE

THE MATHEMATICS OF DATA STRUCTURES & ALGORITHMS

Understanding *Why* Algorithms Work,
Not Just *How* to Code Them

No code. No syntax. Only concepts, notation, and the
reasoning that lets you design efficient algorithms
from first principles.

A First-Principles Mathematical Treatment of DSA

Contents

Preface: How to Read This Book

vii

I Mathematical Foundations 1

1 Why Math? The Language of Efficiency 3

1.1 Two Programs, One Question 3

1.2 Counting, Not Timing 3

1.3 The Three Questions Every Data Structure Answers 4

2 Growth Rates and Asymptotic Notation 7

2.1 The Idea Before the Symbols 7

2.2 Big-O: An Upper Bound 7

2.3 Big-Omega: A Lower Bound 8

2.4 Big-Theta: A Tight Bound 9

2.5 Little-o and Little-omega: Strict Bounds 9

2.6 The Hierarchy of Growth Rates 10

2.7 Best, Worst, and Average Case 11

3 Summations: The Mathematics of Loops 13

3.1 Why Summations Matter 13

3.2 The Arithmetic Series 13

3.3 The Geometric Series 14

3.4 The Harmonic Series 15

3.5 Manipulating Sums: Splitting and Bounding 15

3.6 Nested Sums: The Mathematics of Nested Loops 16

4 Logarithms and Recurrence Relations 19

4.1 What a Logarithm Really Counts 19

4.2	Recurrence Relations: Defining a Function in Terms of Itself	20
4.2.1	The Recursion Tree Method	20
4.2.2	The Substitution Method (Guess and Verify)	21
4.2.3	The Master Theorem	22
5	Combinatorics and Probability for Algorithms	25
5.1	Counting: Permutations and Combinations	25
5.2	Basic Probability	26
5.3	The Pigeonhole Principle	27
5.4	The Birthday Paradox	28
6	Proof Techniques Every Algorithm Designer Needs	29
6.1	Mathematical Induction	29
6.2	Loop Invariants: Induction for Iterative Algorithms	30
6.3	Proof by Contradiction	31
6.4	The Exchange Argument: Proving Greedy Algorithms Correct	31
II	Linear Data Structures	33
7	Arrays: The Mathematics of Contiguous Memory	35
7.1	An Array Is a Linear Function	35
7.2	Two-Dimensional Arrays: Row-Major Addressing	36
7.3	The Cost of Insertion: A Shifting Argument	36
7.4	Dynamic Arrays and the Doubling Strategy	37
8	Linked Lists: The Mathematics of Pointer Chains	39
8.1	No Formula, Only a Chain	39
8.2	Counting Pointers: A Structural Argument	40
8.3	The Mathematics of Cycle Detection	40
9	Stacks and Queues: Order as a Mathematical Constraint	43
9.1	Stacks: Last-In-First-Out as a Formal Rule	43
9.2	Queues: First-In-First-Out and Modular Arithmetic	44
9.2.1	The Circular Buffer: Modular Arithmetic in Action	44
9.3	Dequeues and the Symmetry of Two-Ended Access	45
9.4	Stacks and Queues as Building Blocks	45

III	Trees	47
10	Tree Foundations: Height, Depth, and Counting Nodes	49
10.1	A Tree Is a Connected, Acyclic Graph	49
10.2	Depth, Height, and Levels	49
10.3	Counting Nodes in a Complete Binary Tree	50
10.4	Traversal Orders as Recursive Definitions	51
10.5	Counting All Possible Tree Shapes	51
11	Binary Search Trees: An Invariant That Enables Search	53
11.1	The BST Invariant	53
11.2	Why Search, Insert, and Delete Are All $O(h)$	53
11.3	Average Height of a Randomly Built BST	54
11.4	Order Statistics: Augmenting the Invariant	55
12	Balanced Trees: Forcing Logarithmic Height	57
12.1	The Balance Invariant	57
12.2	Proving AVL Height Is $\Theta(\log n)$ — The Fibonacci Connection	57
12.3	Rotations: Restoring the Invariant in $O(1)$	58
12.4	Red-Black Trees: A Looser, Cheaper Invariant	59
13	Heaps: A Weaker Invariant, Array-Packed	61
13.1	The Heap Property	61
13.2	The Array-Index Formula for a Complete Tree	61
13.3	Sift-Up and Sift-Down: Cost Analysis	62
13.4	Building a Heap from Scratch: Why It's $\Theta(n)$, Not $\Theta(n \log n)$	63
13.5	Heap Sort: Reusing the Heap Invariant	64
14	Tries: Trading Key Length for Key Count	65
14.1	An Invariant Based on Shared Prefixes	65
14.2	Cost Analysis	65
14.3	The Space–Time Trade: Counting Nodes	66
IV	Hashing	69
15	Hashing: Turning Keys into Addresses	71
15.1	The Core Idea: Simulate an Array Formula for Arbitrary Keys	71
15.2	Why Collisions Are Mathematically Unavoidable	71

15.3	Designing a Hash Function: The Division and Multiplication Methods	72
15.4	Load Factor: The Central Health Metric	72
15.4.1	Chaining: Expected Cost as a Direct Function of α	73
15.4.2	Open Addressing: Probe Sequences	73
15.5	Resizing: Reusing the Amortized Argument	74
15.6	Universal Hashing: Defeating Adversarial Inputs	74
V	Graphs	77
16	Graph Foundations and Traversal	79
16.1	Graphs Generalize Trees	79
16.2	The Handshake Lemma	79
16.3	Two Representations, Two Cost Profiles	80
16.4	Breadth-First Search: Layered by Distance	80
16.5	Depth-First Search: Stack Order and Edge Classification	81
17	Shortest Paths: Relaxation as a Mathematical Operation	83
17.1	The Relaxation Operation	83
17.2	Dijkstra's Algorithm: A Greedy Proof	83
17.3	Bellman-Ford: Trading Speed for Generality	85
17.4	Floyd-Warshall: All-Pairs via Dynamic Programming	86
18	Minimum Spanning Trees: The Cut Property	87
18.1	The Problem, Formally	87
18.2	The Cut Property: The Master Theorem of MSTs	87
18.3	Kruskal's Algorithm: Sort, Then Greedily Union	88
18.4	Prim's Algorithm: Grow One Tree from a Seed	89
VI	Algorithm Design Paradigms	91
19	Divide and Conquer: Recursion as a Design Strategy	93
19.1	The Three-Step Template	93
19.2	Merge Sort, Revisited as a Design Choice	93
19.3	Quickselect: Divide and Conquer Without Combining	94
19.4	Strassen's Algorithm: Reducing the Branching Factor	95
20	Dynamic Programming: Recurrences with Memory	97

20.1	The Two Mathematical Preconditions	97
20.2	Diagnosing Overlap: A Counting Argument	97
20.3	Two Implementations, One Recurrence	98
20.4	Worked Example: 0/1 Knapsack	99
20.5	Reconstructing the Solution, Not Just the Value	100
20.6	DP on a Different Axis: Longest Common Subsequence	100
21	Greedy Algorithms: When Local Optimality Suffices	103
21.1	The Greedy Template, and Why It's Dangerous	103
21.2	The Matroid Abstraction (Informally)	103
21.3	Worked Proof: Fractional Knapsack	104
21.4	Huffman Coding: Greedy Merging	105
21.5	When Greedy Fails: A Cautionary Counterexample	106
22	Backtracking and Branch and Bound: Pruning the Search Tree	107
22.1	The State Space Tree	107
22.2	Pruning as Reducing the Effective Branching Factor	108
22.3	Branch and Bound: Pruning by a Numerical Bound	108
22.4	Backtracking's Relationship to Recursion and Induction	109
VII	Advanced Analysis	111
23	Amortized Analysis: Averaging Over Time, Rigorously	113
23.1	Why Worst-Case-Per-Operation Can Be Too Pessimistic	113
23.2	Method 1: Aggregate Analysis	113
23.3	Method 2: The Accounting Method	114
23.4	Method 3: The Potential Method	115
24	Randomized Algorithms: Probability as a Design Tool	117
24.1	Two Flavors of Randomization	117
24.2	Revisiting Randomized Quicksort's Guarantee	117
24.3	Randomized Algorithms as a Tool for Simplicity: Quickselect	117
24.4	Monte Carlo Example: Randomized Primality-Style Verification	118
24.5	The General Lesson: Randomization Buys Simplicity or Speed	119
25	P, NP, and the Limits of Efficient Computation	121
25.1	Defining the Classes	121
25.2	Reductions: The Mathematics of "At Least As Hard"	122

25.3 A Worked Reduction: Vertex Cover from 3-SAT (Sketch)	122
25.4 What to Do When You Suspect NP-Hardness	123

VIII Synthesis 125

26 A Framework for Designing Efficient Algorithms	127
26.1 Step 1: Characterize the Problem Mathematically	127
26.2 Step 2: Choose an Invariant	127
26.3 Step 3: If Recursive, Write the Recurrence and Classify It	128
26.4 Step 4: Decide Worst-Case, Average-Case, or Amortized Is the Right Lens	128
26.5 Step 5: Check for a Lower Bound Before Chasing an Impossible Speedup	129
26.6 A Worked Walkthrough of the Framework	129
27 Complexity Cheat Sheet	131
27.1 Data Structure Operations	131
27.2 Sorting Algorithms	131
27.3 Graph Algorithms	132
27.4 The Master Theorem, One More Time	132
27.5 Growth Rate Hierarchy (Chapter 2)	132
27.6 The Proof Techniques Toolbox (Chapter 6)	133
27.7 Closing Words	133

Preface: How to Read This Book

Most people learn data structures and algorithms backwards. They learn a syntax first, memorize that “quicksort is fast,” and only years later, if ever, ask *why* $n \log n$ is the natural speed limit for comparison sorting, or *why* a balanced tree must have height $O(\log n)$ and cannot do better.

This book inverts that order. There is no code anywhere in these pages. Instead, every data structure and every algorithm is treated as what it actually is underneath: a piece of mathematics that happens to be executable. An array is a linear function from indices to memory addresses. A hash table is an application of the pigeonhole principle and probability theory. A balanced tree is a proof that height and node count can be forced into a logarithmic relationship. Dynamic programming is nothing more than a recurrence relation evaluated in the right order. Once you see the mathematics, the design of new algorithms stops being a matter of memorizing patterns and becomes a matter of *asking the right structural questions*.

The Promise of This Book

By the final chapter, you should be able to look at a new problem and instinctively ask:

- What is the state space, and how large is it?
- Is there a recurrence hiding in this problem? Does it overlap with itself?
- What invariant could I maintain to keep an operation cheap?
- Can I trade memory for time, or exactness for expected-case speed?
- What is the information-theoretic lower bound — can I even hope to do better?

These are mathematical questions, and this book gives you the mathematical vocabulary to answer them.

How Each Chapter Is Built

Every chapter follows the same rhythm:

- **Definition boxes** state a concept with precise, simple notation.
- **Step-by-step derivation boxes** build a formula from scratch — never a formula pulled out of thin air.
- **Worked examples** apply the math to small, concrete numbers you can verify by hand.
- **Key insight boxes** connect the math back to the intuition of *why it matters for design*.
- **Takeaway lines** close every section with the one sentence worth remembering.

The Only Notation You Need

This book intentionally uses a small, consistent set of symbols so you are never re-learning notation:

Symbol	Meaning
n	size of the input (number of elements)
$T(n)$	running time as a function of n
$O(\cdot), \Omega(\cdot), \Theta(\cdot)$	asymptotic upper, lower, tight bounds
$\log n$	logarithm, base 2 unless stated otherwise
$\sum$	sum over a range
$\lfloor x \rfloor, \lceil x \rceil$	floor and ceiling of x
$\Pr[\cdot]$	probability of an event
$\mathbb{E}[\cdot]$	expected value

Keep this table in mind; nothing more exotic appears in this book. Let's begin with the single most important question in all of algorithm design: what does it even mean for one algorithm to be “faster” than another?

Part I

Mathematical Foundations

CHAPTER 1

Why Math? The Language of Efficiency

“An algorithm must be seen to be believed.” — Donald Knuth. But before you can see it, you must be able to count it.

1.1 Two Programs, One Question

Imagine two procedures that both find whether a value exists in a list of n numbers. One checks every number, one at a time. The other first requires the list to be sorted, then repeatedly cuts the search space in half. Both procedures are correct. Both give the right answer. Yet if n is a billion, one finishes in the blink of an eye and the other takes noticeably long.

The difference between these two procedures cannot be measured with a stopwatch alone, because a stopwatch depends on the speed of the machine, the programming language, and even how busy the computer is that day. What we need is a machine-independent, language-independent way to describe *how the amount of work grows as the input grows*. That is precisely what mathematics gives us, and nothing else can.

KEY INSIGHT

Efficiency is not a property of code. It is a property of a *function* — the function that maps input size n to the number of basic operations performed. Everything in this book is really the study of that one function, $T(n)$, and how to make it grow as slowly as possible.

1.2 Counting, Not Timing

To analyze an algorithm mathematically we do not measure seconds; we count *elementary steps*: a comparison, an addition, a memory access, a pointer follow. If an algorithm performs $c_1n^2 + c_2n + c_3$ such steps for some constants c_1, c_2, c_3 , we care about the shape of that function — a quadratic — far more than the exact

constants, because the constants depend on the machine, but the shape depends only on the algorithm's logic.

WORKED EXAMPLE Linear Scan vs. Halving Search

Suppose $n = 1,000,000$.

- A linear scan performs, in the worst case, $n = 1,000,000$ comparisons.
- A halving search performs, in the worst case, $\lceil \log_2 n \rceil = 20$ comparisons.

The halving search is roughly 50,000 times fewer comparisons. This is not a constant-factor engineering trick — it is a structural, mathematical difference in the shape of the two functions n versus $\log n$. No amount of faster hardware for the linear scan can catch up, because doubling n doubles the linear scan's work, but only *adds one step* to the halving search.

1.3 The Three Questions Every Data Structure Answers

Every data structure in this book can be understood as a mathematical answer to three questions:

1. **What shape does the data live in?** (a straight line, a tree, a graph, a set of buckets)
2. **What invariant does the structure maintain?** (sorted order, balanced height, heap property, prefix compression)
3. **What is the cost, as a function of n , of preserving that invariant while reading or modifying the data?**

The entire discipline of data structure design is a negotiation between question 2 and question 3: the more structure (invariant) you maintain, the more it typically costs to update, but the cheaper it becomes to query. This trade-off is the mathematical heartbeat of this whole book, and you will see it again in every single chapter.

DEFINITION Invariant

An **invariant** is a mathematical condition that a data structure guarantees is true before and after every operation. For example, “the array is sorted” or

“every node’s key is greater than its left child’s key and less than its right child’s key.” Algorithms are designed by first choosing an invariant, and then proving that every operation preserves it.

TAKEAWAY: *Designing an algorithm is really the act of choosing an invariant to maintain and then computing the cost, as a function of n , of maintaining it.*

CHAPTER 2

Growth Rates and Asymptotic Notation

2.1 The Idea Before the Symbols

Before writing a single formula, understand the idea in plain words: we want to describe how a function $T(n)$ behaves when n becomes very large, while ignoring two things that don't matter for large-scale comparisons — constant multipliers, and what happens for small n . Asymptotic notation is simply a formal bookkeeping system for making statements like “this function eventually grows no faster than that one.”

2.2 Big-O: An Upper Bound

DEFINITION Big-O Notation

Let $f(n)$ and $g(n)$ be functions of a positive integer n . We say

$$f(n) = O(g(n))$$

if there exist positive constants c and n_0 such that

$$0 \leq f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0.$$

In words: from some point n_0 onward, $f(n)$ never exceeds a constant multiple of $g(n)$. $O(g(n))$ describes a **ceiling** on growth.

STEP-BY-STEP DERIVATION Proving $3n^2 + 5n + 2 = O(n^2)$

We must find constants c, n_0 satisfying the definition.

$$\begin{aligned} 3n^2 + 5n + 2 &\leq 3n^2 + 5n^2 + 2n^2 \\ &= 10n^2 \end{aligned}$$

(The first step holds whenever $n \geq 1$, since $n \leq n^2$ and $1 \leq n^2$ for all $n \geq 1$.) So choosing $c = 10$ and $n_0 = 1$ satisfies $3n^2 + 5n + 2 \leq c \cdot n^2$ for all $n \geq n_0$. Hence $3n^2 + 5n + 2 = O(n^2)$. Notice we threw away the smaller terms $5n$ and 2 entirely — that is exactly the point of the notation: only the *dominant* term, n^2 , matters at scale.

KEY INSIGHT

A useful shortcut: to find the Big-O of a polynomial, keep only the highest-power term and drop its coefficient. $7n^3 - 2n^2 + 9 = O(n^3)$. This works because for large n , any higher power of n eventually dwarfs any lower power and any constant, no matter how large the lower-order coefficients are.

2.3 Big-Omega: A Lower Bound

DEFINITION Big-Omega Notation

$$f(n) = \Omega(g(n))$$

if there exist positive constants c and n_0 such that

$$0 \leq c \cdot g(n) \leq f(n) \quad \text{for all } n \geq n_0.$$

This is the mirror image of Big-O: it says $f(n)$ grows *at least as fast* as $g(n)$ — a **floor** on growth.

2.4 Big-Theta: A Tight Bound

DEFINITION Big-Theta Notation

$$f(n) = \Theta(g(n))$$

if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$ simultaneously. Equivalently, there exist positive constants c_1, c_2, n_0 such that

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \text{for all } n \geq n_0.$$

Θ pins $f(n)$ between a floor and a ceiling of the *same* shape — this is the notation you want whenever you can, because it is the most informative.

WORKED EXAMPLE Classifying a Real Running-Time Function

Consider an algorithm whose exact operation count is $T(n) = 4n^2 + 2n \log n + 100$.

- Dominant term: $4n^2$.
- Upper bound: $T(n) = O(n^2)$, since $2n \log n$ and 100 are both eventually smaller than n^2 .
- Lower bound: $T(n) = \Omega(n^2)$, since $T(n) \geq 4n^2$ for all $n \geq 1$.
- Because both hold: $T(n) = \Theta(n^2)$.

2.5 Little-o and Little-omega: Strict Bounds

DEFINITION Little-o and Little-omega

$f(n) = o(g(n))$ means f grows *strictly slower* than g : for every constant $c > 0$, there is an n_0 such that $f(n) < c \cdot g(n)$ for all $n \geq n_0$. Equivalently,

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0.$$

Symmetrically, $f(n) = \omega(g(n))$ means f grows *strictly faster* than g , i.e. $\lim_{n \rightarrow \infty} f(n)/g(n) = \infty$.

The limit-of-the-ratio test above is, in practice, the fastest way to compare two functions:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \begin{cases} 0 & \Rightarrow f = o(g), \text{ so } f = O(g) \text{ but } f \neq \Theta(g) \\ c > 0 \text{ (constant)} & \Rightarrow f = \Theta(g) \\ \infty & \Rightarrow f = \omega(g), \text{ so } f = \Omega(g) \text{ but } f \neq \Theta(g) \end{cases}$$

WORKED EXAMPLE Comparing $n \log n$ and $n^{1.5}$

Compute $\lim_{n \rightarrow \infty} \frac{n \log n}{n^{1.5}} = \lim_{n \rightarrow \infty} \frac{\log n}{\sqrt{n}}$. As $n \rightarrow \infty$, $\sqrt{n}$ outgrows $\log n$ (proved rigorously via L'Hopital's rule or the standard fact that every polynomial eventually beats every logarithm), so this limit is 0. Hence $n \log n = o(n^{1.5})$: merge sort's $\Theta(n \log n)$ is asymptotically faster than any $\Theta(n^{1.5})$ algorithm, even though for small n the constants could make either one faster in practice.

2.6 The Hierarchy of Growth Rates

Memorize this ladder — it lets you rank almost any algorithm at a glance, from best to worst:

$$1 < \log n < \log^2 n < \sqrt{n} < n < n \log n \\ < n^2 < n^3 < 2^n < n! < n^n$$

Growth class	$n = 10$	$n = 100$	$n = 1,000$	$n = 10^6$
$\log_2 n$	3.3	6.6	10.0	20.0
n	10	100	1,000	10^6
$n \log_2 n$	33	664	9,966	2×10^7
n^2	100	10,000	10^6	10^{12}
2^n	1,024	1.3×10^{30}	astronomically large	—

Table 2.1: How dramatically growth rates separate as n scales. This table alone explains why exponential-time algorithms are unusable beyond tiny inputs.

KEY INSIGHT

This is why asymptotic notation is not academic pedantry: it *predicts* whether your program will finish in a second or in longer than the age of the universe, purely from the shape of $T(n)$, without ever running it. A $\Theta(2^n)$ algorithm at $n = 100$ requires more operations than there are atoms in the observable universe.

2.7 Best, Worst, and Average Case

Asymptotic notation describes a function, but which function? An algorithm can behave differently on different inputs of the same size n . We distinguish:

- **Worst case:** $T_{\text{worst}}(n) = \max_{\text{inputs of size } n} T(\text{input})$. This is what $O(\cdot)$ conventionally bounds in most textbooks, and gives an ironclad guarantee.
- **Best case:** $T_{\text{best}}(n) = \min_{\text{inputs of size } n} T(\text{input})$. Rarely a useful guarantee, since adversarial or even ordinary inputs may never hit it.
- **Average case:** $T_{\text{avg}}(n) = \mathbb{E}[T(\text{input})]$, an expectation over a probability distribution of inputs (often “all permutations equally likely”). This requires the probability tools of Chapter 5.

WORKED EXAMPLE Linear Search's Three Faces

Searching for a target in an unsorted array of n elements by scanning left to right:

- Best case: target is first element $\Rightarrow$ 1 comparison $\Rightarrow \Theta(1)$.
- Worst case: target is last element or absent $\Rightarrow n$ comparisons $\Rightarrow \Theta(n)$.
- Average case (target present, uniformly at a random position): expected position is $\frac{n+1}{2} \Rightarrow \Theta(n)$ still, but with half the constant.

TAKEAWAY: Big-O gives a ceiling, Big-Omega a floor, Big-Theta a tight fit — and always ask which case (best, worst, average) a claimed bound refers to, because the same algorithm can have wildly different bounds under each.

CHAPTER 3

Summations: The Mathematics of Loops

3.1 Why Summations Matter

Every loop in every algorithm is, mathematically, a summation. A single loop that runs n times and does constant work per iteration sums to $\sum_{i=1}^n 1 = n$. A nested loop, where the inner loop's range depends on the outer variable, produces far more interesting sums — and these sums are exactly how we derive formulas like “insertion sort is $\Theta(n^2)$ ” from first principles, rather than by memorization.

3.2 The Arithmetic Series

DEFINITION Arithmetic Series

For constants that increase by a fixed amount each step,

$$\sum_{i=1}^n i = 1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}.$$

STEP-BY-STEP DERIVATION Gauss's Pairing Trick

Write the sum forwards and backwards, and add the two copies term by term:

$$\begin{array}{r} S = 1 + 2 + \cdots + (n-1) + n \\ S = n + (n-1) + \cdots + 2 + 1 \\ \hline 2S = (n+1) + (n+1) + \cdots + (n+1) \quad (n \text{ copies of } n+1) \\ 2S = n(n+1) \\ S = \frac{n(n+1)}{2} \end{array}$$

This elegant trick, attributed to a young Gauss, generalizes to any arithmetic series and is the single most-used identity in algorithm analysis.

WORKED EXAMPLE Selection Sort's Inner Loop

Selection sort's outer loop runs for $i = 1$ to n , and on iteration i it scans the remaining $n - i$ elements to find the minimum. Total comparisons:

$$\sum_{i=1}^{n-1} (n - i) = (n - 1) + (n - 2) + \cdots + 1 + 0 = \frac{(n - 1)n}{2} = \Theta(n^2).$$

This single sum *is* the proof that selection sort is quadratic — not an assertion to memorize, but a direct consequence of counting a triangular sum.

3.3 The Geometric Series

DEFINITION Geometric Series

For a fixed ratio $r \neq 1$,

$$\sum_{i=0}^{n-1} r^i = 1 + r + r^2 + \cdots + r^{n-1} = \frac{r^n - 1}{r - 1}.$$

For an infinite series with $|r| < 1$: $\sum_{i=0}^{\infty} r^i = \frac{1}{1 - r}$.

STEP-BY-STEP DERIVATION Deriving the Finite Geometric Sum

Let $S = 1 + r + r^2 + \cdots + r^{n-1}$. Multiply both sides by r :

$$rS = r + r^2 + \cdots + r^{n-1} + r^n.$$

Subtract the original from this:

$$rS - S = r^n - 1 \implies S(r - 1) = r^n - 1 \implies S = \frac{r^n - 1}{r - 1}.$$

WORKED EXAMPLE Why the Levels of a Binary Tree Sum to About $2n$

A complete binary tree with n leaves has $\log_2 n$ levels, and level k (counting from the root as level 0) has 2^k nodes. The total number of nodes is

$$\sum_{k=0}^{\log_2 n} 2^k = \frac{2^{\log_2 n + 1} - 1}{2 - 1} = 2n - 1.$$

This single geometric-series fact explains why a binary tree storing n leaves has almost exactly $2n$ total nodes — no more, no less — and is the reason recursive divide-and-conquer algorithms that do constant work per node run in $\Theta(n)$ total across all levels, while doing $O(\log n)$ work *per level* typically yields an $O(n \log n)$ total (see Chapter 19).

3.4 The Harmonic Series

DEFINITION Harmonic Series

$$H_n = \sum_{i=1}^n \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \approx \ln n + \gamma$$

where $\gamma \approx 0.5772$ is the Euler–Mascheroni constant. In asymptotic terms, $H_n = \Theta(\log n)$.

KEY INSIGHT

The harmonic series is the hidden reason several classic algorithms have a $\log n$ factor even though nothing about them looks like a binary halving process. For instance, in randomized quickselect and in the analysis of hashing with chaining, sums of the form $\sum 1/i$ appear directly and are what produce the logarithmic term.

3.5 Manipulating Sums: Splitting and Bounding

Two techniques handle almost every sum you will meet in algorithm analysis:

1. Splitting a sum into known pieces.

$$\sum_{i=1}^n (3i + 5) = 3 \sum_{i=1}^n i + \sum_{i=1}^n 5 = 3 \cdot \frac{n(n+1)}{2} + 5n.$$

2. Bounding a sum by an integral (for approximate analysis). If f is increasing,

$$\int_0^n f(x) dx \leq \sum_{i=1}^n f(i) \leq \int_1^{n+1} f(x) dx.$$

WORKED EXAMPLE Bounding $\sum_{i=1}^n \log i$

Using the integral bound with $f(x) = \log x$:

$$\sum_{i=1}^n \log i \approx \int_1^n \log x dx = n \log n - n + 1 = \Theta(n \log n).$$

This is precisely the mathematical fact behind why computing the product $1 \times 2 \times \cdots \times n$ digit-by-digit, or evaluating $\log(n!)$ via Stirling's approximation, gives an $n \log n$ -shaped bound — the same shape that appears in every optimal comparison sort, which is not a coincidence, as Chapter 19 will prove.

3.6 Nested Sums: The Mathematics of Nested Loops

STEP-BY-STEP DERIVATION A Double Sum for a Triangular Loop

Consider an outer loop i from 1 to n , with an inner loop j from 1 to i , doing constant work each time:

$$\sum_{i=1}^n \sum_{j=1}^i 1 = \sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2).$$

Even though the inner loop does not always run n times, the total work is still quadratic — half of the full $n \times n$ grid, but half of a quadratic quantity is still $\Theta(n^2)$, since constant factors vanish under asymptotic notation.

TAKEAWAY: Whenever you see a loop, write its summation. Whenever you see nested loops,

write a nested summation. The closed form of that summation is the algorithm's time complexity — this is not an analogy, it is a direct equivalence.

CHAPTER 4

Logarithms and Recurrence Relations

4.1 What a Logarithm Really Counts

DEFINITION Logarithm

$\log_b n$ is the answer to the question: “how many times must I multiply 1 by b to reach n ?” Equivalently, $b^{\log_b n} = n$. In computer science, when the base is omitted, $\log n$ means $\log_2 n$, because so many algorithms repeatedly *halve* their input.

KEY INSIGHT

Every time you see a logarithm appear in an algorithm’s complexity, ask: “what quantity is being repeatedly divided by a constant factor until it reaches 1?” That question, answered honestly, is the entire reason binary search, balanced trees, and divide-and-conquer algorithms all have a $\log n$ term. $\log_2 n$ literally counts “how many times can I halve n before I hit 1.”

STEP-BY-STEP DERIVATION Why Binary Search Takes $\lceil \log_2 n \rceil$ Steps

Let $f(n)$ be the number of comparisons binary search needs in the worst case on n elements. Each comparison discards half the remaining elements:

$$f(n) = f(n/2) + 1, \quad f(1) = 1.$$

Unroll the recursion: $f(n) = f(n/2) + 1 = f(n/4) + 2 = f(n/8) + 3 = \dots = f(n/2^k) + k$. The recursion bottoms out when $n/2^k = 1$, i.e. $k = \log_2 n$. So

$$f(n) = f(1) + \log_2 n = 1 + \log_2 n = \Theta(\log n).$$

4.2 Recurrence Relations: Defining a Function in Terms of Itself

DEFINITION Recurrence Relation

A recurrence expresses $T(n)$ — the cost on an input of size n — in terms of T applied to smaller inputs, plus some extra work done at this level. General divide-and-conquer shape:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

where a is the number of subproblems, n/b is the size of each subproblem, and $f(n)$ is the cost of dividing and combining.

There are three standard techniques for solving recurrences: the **recursion tree method**, the **substitution method**, and the **Master Theorem** (a ready-made formula for the shape above). Let's build up all three.

4.2.1 The Recursion Tree Method

The idea: draw the recursive calls as a tree, compute the total work done *at each level*, then sum across all levels.

WORKED EXAMPLE Merge Sort's Recurrence via a Tree

Merge sort splits an array of size n into 2 halves and spends $\Theta(n)$ time merging them back:

$$T(n) = 2T(n/2) + cn.$$

Level 0 (root): 1 problem of size n , cost cn .

Level 1: 2 problems of size $n/2$, cost $2 \cdot c(n/2) = cn$.

Level 2: 4 problems of size $n/4$, cost $4 \cdot c(n/4) = cn$.

Level k : 2^k problems of size $n/2^k$, cost $2^k \cdot c(n/2^k) = cn$.

Every level costs exactly cn ! The tree has $\log_2 n$ levels (since the problem size shrinks from n to 1 by halving, as derived above). Total cost:

$$T(n) = \underbrace{cn + cn + \cdots + cn}_{\log_2 n + 1 \text{ levels}} = cn(\log_2 n + 1) = \Theta(n \log n).$$

KEY INSIGHT

The recursion tree method turns a scary recurrence into simple arithmetic: (cost per level) $\times$ (number of levels). Whenever the cost per level is constant across levels (as in merge sort), the total is simply that cost times the tree's height.

4.2.2 The Substitution Method (Guess and Verify)

STEP-BY-STEP DERIVATION Verifying $T(n) = 2T(n/2) + n \Rightarrow T(n) = O(n \log n)$ by Induction

Guess: $T(n) \leq cn \log n$ for some constant c and all $n \geq n_0$.

Inductive hypothesis: assume $T(n/2) \leq c(n/2) \log(n/2)$ holds for the smaller input.

Inductive step: substitute into the recurrence.

$$\begin{aligned}
 T(n) &= 2T(n/2) + n \\
 &\leq 2 \left[c \frac{n}{2} \log \frac{n}{2} \right] + n \\
 &= cn \log \frac{n}{2} + n \\
 &= cn(\log n - 1) + n \\
 &= cn \log n - cn + n \\
 &= cn \log n - (c - 1)n
 \end{aligned}$$

This is $\leq cn \log n$ exactly when $(c - 1)n \geq 0$, i.e. whenever $c \geq 1$. Choosing $c = 1$ (and checking the base case directly) completes the induction, proving $T(n) = O(n \log n)$.

KEY INSIGHT

Substitution is the rigorous way to *prove* a bound you already suspect (often from a recursion tree sketch). It is the algorithmic-analysis analogue of mathematical induction: assume the bound holds for smaller inputs, then show that assumption forces the bound to hold for the current input.

4.2.3 The Master Theorem

For recurrences of the shape $T(n) = aT(n/b) + f(n)$ with $a \geq 1, b > 1$, compare $f(n)$ against $n^{\log_b a}$ — the cost that the *recursive branching alone* would produce if the extra work $f(n)$ were zero.

DEFINITION Master Theorem — Three Cases

Let $n^{\log_b a}$ be the “watershed” function.

- **Case 1 (leaves dominate):** if $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$, then

$$T(n) = \Theta(n^{\log_b a}).$$

- **Case 2 (balanced):** if $f(n) = \Theta(n^{\log_b a})$, then

$$T(n) = \Theta(n^{\log_b a} \log n).$$

- **Case 3 (root dominates):** if $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$, and the regularity condition $a f(n/b) \leq k f(n)$ holds for some $k < 1$ and large n , then

$$T(n) = \Theta(f(n)).$$

KEY INSIGHT

Intuition for the three cases: compute $n^{\log_b a}$, the amount of work that would exist just from the branching structure of the recursion tree with zero extra work per node. Then ask whether the actual per-level cost $f(n)$ is smaller (Case 1, so the many tiny leaves dominate the total), equal (Case 2, so every level contributes equally, exactly like merge sort), or larger (Case 3, so the very first, top-level call already dominates everything below it).

WORKED EXAMPLE Applying the Master Theorem to Three Classic Recurrences

Binary search: $T(n) = T(n/2) + \Theta(1)$. Here $a = 1, b = 2, f(n) = \Theta(1)$. Watershed: $n^{\log_2 1} = n^0 = 1$. Since $f(n) = \Theta(1) = \Theta(n^0)$, this is Case 2: $T(n) = \Theta(n^0 \log n) = \Theta(\log n)$. ✓ (Matches our earlier direct derivation.)

Merge sort: $T(n) = 2T(n/2) + \Theta(n)$. Here $a = 2, b = 2, f(n) = \Theta(n)$. Watershed: $n^{\log_2 2} = n^1 = n$. Since $f(n) = \Theta(n^1)$, Case 2 applies: $T(n) = \Theta(n \log n)$. ✓

Naive matrix multiplication recursively split into 8 subproblems: $T(n) = 8T(n/2) + \Theta(n^2)$. Watershed: $n^{\log_2 8} = n^3$. Since $f(n) = \Theta(n^2) = O(n^{3-\epsilon})$ for $\epsilon = 1$, Case 1 applies: $T(n) = \Theta(n^3)$ — no better than the straightforward triple-nested-loop algorithm, which is exactly why Strassen's algorithm (Chapter 19) had to reduce the number of subproblems from 8 to 7 to gain any advantage.

STEP-BY-STEP DERIVATION Why the Watershed Is $n^{\log_b a}$ — Counting Leaves

In the recursion tree for $T(n) = aT(n/b) + f(n)$ (ignoring $f(n)$ for a moment), the tree branches into a children at every node, and shrinks the problem size by a factor of b at every level. The tree bottoms out (problem size 1) at level k where $n/b^k = 1$, i.e. $k = \log_b n$. The number of leaves at that bottom level is $a^k = a^{\log_b n}$. Using the identity $a^{\log_b n} = n^{\log_b a}$ (provable by taking $\log_b$ of both sides), the total number of leaves — and hence the total base-case work if $f(n) = 0$ — is exactly $n^{\log_b a}$. This is why it is the natural yardstick against which to measure the per-level cost $f(n)$.

TAKEAWAY: Every divide-and-conquer recurrence is a race between the branching factor of the recursion tree ($n^{\log_b a}$) and the cost of combining results at each level ($f(n)$) — whichever one grows faster asymptotically determines the total running time.

CHAPTER 5

Combinatorics and Probability for Algorithms

Many of the most important data structures — hash tables above all — are not designed to guarantee good performance on *every* input, but to guarantee good performance *on average*, or *with high probability*. To reason about that, we need a small, focused toolkit of counting and probability.

5.1 Counting: Permutations and Combinations

DEFINITION Permutations and Combinations

The number of ways to arrange n distinct items is $n! = n \times (n - 1) \times \cdots \times 1$.
The number of ways to choose an ordered sequence of k items from n is

$$P(n, k) = \frac{n!}{(n - k)!}.$$

The number of ways to choose an *unordered* subset of k items from n is

$$\binom{n}{k} = \frac{n!}{k!(n - k)!}.$$

WORKED EXAMPLE Why Comparison Sorting Cannot Beat $n \log n$

There are $n!$ possible orderings of n distinct elements, and a correct sorting algorithm must be able to distinguish between all of them. A comparison-based algorithm is a binary decision tree: each comparison has 2 outcomes, so a tree of depth d can distinguish at most 2^d outcomes. To distinguish $n!$ possible orderings we need

$$2^d \geq n! \implies d \geq \log_2(n!).$$

Using Stirling's approximation, $\log_2(n!) = \Theta(n \log n)$. This proves, from pure counting, that **no comparison-based sort can ever beat $\Omega(n \log n)$ in the worst case** — merge sort and heap sort are therefore asymptotically optimal, not just fast.

STEP-BY-STEP DERIVATION Stirling's Approximation, Informally

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \implies \log_2(n!) \approx n \log_2 n - n \log_2 e = \Theta(n \log n).$$

The key takeaway is not the precise constant, but that $\log(n!)$ has the exact same asymptotic shape as $n \log n$ — this single fact is the reason the $n \log n$ barrier appears throughout comparison-based algorithms.

5.2 Basic Probability

DEFINITION Probability and Expectation

For a discrete random variable X that takes value x_i with probability p_i , the **expected value** is

$$\mathbb{E}[X] = \sum_i x_i \cdot p_i.$$

DEFINITION Linearity of Expectation

For any random variables $X_1, X_2, \dots, X_k$ — even if they are *dependent* on each other —

$$\mathbb{E}[X_1 + X_2 + \dots + X_k] = \mathbb{E}[X_1] + \mathbb{E}[X_2] + \dots + \mathbb{E}[X_k].$$

KEY INSIGHT

Linearity of expectation is the single most powerful trick in average-case algorithm analysis, because it lets us compute the expected cost of a complicated random process by breaking it into simple pieces, analyzing each piece in isolation, and adding the results — with no need to worry about how the pieces interact.

WORKED EXAMPLE Expected Number of Comparisons in Randomized Quicksort

Let X_{ij} be an indicator random variable that is 1 if elements at sorted ranks i and j are ever compared during randomized quicksort, and 0 otherwise. A classical argument shows that ranks i and j are compared exactly when, among the elements with ranks between i and j inclusive, either rank i or rank j is chosen as a pivot first — which happens with probability $\frac{2}{j-i+1}$. By linearity of expectation, the expected total number of comparisons is

$$\mathbb{E} \left[\sum_{i < j} X_{ij} \right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1}.$$

Substituting $k = j - i$ and using the harmonic series from Chapter 3, this sum evaluates to $\Theta(n \log n)$ — proving that randomized quicksort's *expected* running time is $\Theta(n \log n)$, regardless of the input order, even though its *worst-case* time is $\Theta(n^2)$.

5.3 The Pigeonhole Principle

DEFINITION Pigeonhole Principle

If n items are placed into m containers and $n > m$, then at least one container holds more than one item. More generally, if $n > k \cdot m$, some container holds more than k items.

KEY INSIGHT

The pigeonhole principle is the entire mathematical justification for why hash collisions are *unavoidable* in general: if you hash more keys than there are table slots, some slot must receive more than one key, no matter how clever the hash function is. Good hash table design (Chapter 15) is therefore not about avoiding collisions — that is mathematically impossible once the number of keys exceeds the number of slots — but about making collisions rare and cheap to resolve.

5.4 The Birthday Paradox

STEP-BY-STEP DERIVATION Probability of at Least One Collision

Suppose we throw k balls independently and uniformly at random into m bins. What is the probability that at least two balls land in the same bin? It's easier to compute the complement: the probability that *all* k balls land in *different* bins.

$$\Pr[\text{no collision}] = \frac{m}{m} \cdot \frac{m-1}{m} \cdot \frac{m-2}{m} \cdots \frac{m-k+1}{m} = \prod_{i=0}^{k-1} \left(1 - \frac{i}{m}\right).$$

Using the approximation $1 - x \approx e^{-x}$ for small x :

$$\Pr[\text{no collision}] \approx \prod_{i=0}^{k-1} e^{-i/m} = e^{-\sum_{i=0}^{k-1} i/m} = e^{-k(k-1)/2m}.$$

So $\Pr[\text{at least one collision}] \approx 1 - e^{-k^2/2m}$, which becomes larger than $1/2$ once $k \approx 1.18\sqrt{m}$ — dramatically smaller than m itself.

WORKED EXAMPLE Why Hash Table Collisions Feel “Too Frequent”

With $m = 365$ “bins” (days of the year), collisions become likely (over 50%) with only $k \approx 23$ people — far fewer than the 365 one might naively expect. The same mathematics tells us that a hash table with $m = 1,000,000$ slots will already have a better-than-even chance of at least one collision after only about 1,177 insertions — roughly $\sqrt{m}$, not m . This is precisely why hash table design must assume collisions will happen early and often, and must have an efficient resolution strategy (Chapter 15) rather than hoping to avoid them.

TAKEAWAY: *Average-case and randomized analysis is not guesswork — it is exact mathematics using expectation, linearity of expectation, and counting arguments like the pigeonhole principle, and it is precisely what justifies why hash tables and randomized quicksort are trusted in practice despite having poor theoretical worst cases.*

CHAPTER 6

Proof Techniques Every Algorithm Designer Needs

Knowing that an algorithm is *fast* is only half the job — you must also know it is *correct*. This chapter builds the three proof techniques that appear, in disguise, throughout the rest of the book: mathematical induction, proof by contradiction, and the exchange argument.

6.1 Mathematical Induction

DEFINITION Mathematical Induction

To prove a statement $P(n)$ holds for all integers $n \geq n_0$:

1. **Base case:** prove $P(n_0)$ directly.
2. **Inductive step:** assume $P(k)$ holds for some arbitrary $k \geq n_0$ (the *inductive hypothesis*), and use it to prove $P(k + 1)$.

If both hold, $P(n)$ is true for every $n \geq n_0$, because the base case starts a chain, and the inductive step shows each link pulls up the next.

KEY INSIGHT

Induction is the mathematical twin of *recursion*. A recursive algorithm's correctness proof is almost always a direct translation of the algorithm's own recursive structure into an inductive proof: the base case of the algorithm becomes the base case of the proof, and "trust the recursive call to work correctly on a smaller input" becomes the inductive hypothesis.

WORKED EXAMPLE Proving Merge Sort Is Correct by Induction

Claim: merge sort correctly sorts any array of n elements.

Base case ($n \leq 1$): an array of 0 or 1 elements is trivially sorted; merge sort returns it unchanged.

Inductive hypothesis: assume merge sort correctly sorts any array of fewer than n elements.

Inductive step: for an array of $n > 1$ elements, merge sort splits it into two halves, each of size $< n$. By the inductive hypothesis, both halves are correctly sorted by the recursive calls. It remains only to show that the *merge* step correctly combines two sorted halves into one sorted array — which follows because at each step, the merge procedure selects the smaller of the two current front elements, and induction on the number of elements merged so far confirms the growing output stays sorted. Hence the whole array is sorted. ■

6.2 Loop Invariants: Induction for Iterative Algorithms

DEFINITION Loop Invariant Proof

A **loop invariant** is a property that is true:

- **Initialization:** before the first iteration begins.
- **Maintenance:** if true before an iteration, it remains true before the next.
- **Termination:** when the loop ends, the invariant (combined with the reason the loop ended) implies the algorithm is correct.

This is exactly mathematical induction, with “before iteration i ” playing the role of $P(i)$.

WORKED EXAMPLE Proving Insertion Sort Correct via a Loop Invariant

Invariant: at the start of each iteration i , the subarray $A[1..i-1]$ consists of the original elements in that range, sorted.

Initialization: before the first iteration, $i = 2$ so $A[1..1]$ is a single element — trivially sorted.

Maintenance: assuming $A[1..i-1]$ is sorted, insertion sort shifts elements to insert $A[i]$ into its correct position within $A[1..i-1]$, so $A[1..i]$ is now sorted,

restoring the invariant for the next iteration.

Termination: the loop ends when $i = n + 1$, so the invariant states $A[1..n]$ is sorted — precisely what we wanted to prove. ■

6.3 Proof by Contradiction

DEFINITION Proof by Contradiction

To prove a statement Q , assume $\neg Q$ (that Q is false), and derive a logical impossibility from that assumption. Since a false assumption cannot lead to a valid impossibility unless the assumption itself was wrong, Q must be true.

WORKED EXAMPLE Proving Binary Search's Range Never Loses the Target

Claim: if the target x is present in a sorted array, binary search never discards the half of the array containing it.

Proof (by contradiction): suppose, on some iteration, binary search discards the half containing x . It only discards the left half when $x > A[\text{mid}]$, and only discards the right half when $x < A[\text{mid}]$. Since the array is sorted, $x > A[\text{mid}]$ implies x can only appear to the right of mid — so discarding the *left* half in this case cannot discard x ; a symmetric argument covers the other case. Either way, assuming binary search discards the half containing x leads to a contradiction of the sortedness of A . Hence binary search never loses the target. ■

6.4 The Exchange Argument: Proving Greedy Algorithms Correct

DEFINITION Exchange Argument

To prove a greedy choice is safe (leads to an optimal solution), take an arbitrary optimal solution O that does *not* make the greedy choice, and show that you can modify (“exchange” a piece of) O to incorporate the greedy choice *without making the solution any worse*. This proves an optimal solution

exists that agrees with the greedy choice, justifying making it first.

WORKED EXAMPLE Activity Selection: Why “Pick the Earliest Finish Time” Is Safe

Problem: given activities with start and finish times, select the maximum number of non-overlapping activities.

Greedy claim: always pick the activity with the earliest finish time first.

Proof by exchange: let a be the activity with the earliest finish time, and let O be an optimal solution. If $a \in O$, the greedy choice already agrees with O . If $a \notin O$, let b be the activity in O with the earliest finish time. Since a has the earliest finish time among *all* activities, $\text{finish}(a) \leq \text{finish}(b)$. Replace b with a in O to form O' . Because a finishes no later than b , a cannot conflict with any activity in O that started after b finished, so O' remains a valid, non-overlapping schedule with the same number of activities as O . Hence O' is also optimal, and it contains the greedy choice a . ■

This single exchange argument is the rigorous mathematical justification behind an entire family of greedy algorithms explored in Chapter 21.

TAKEAWAY: *Correctness proofs are not an afterthought bolted onto a fast algorithm — induction mirrors recursion, loop invariants mirror iteration, and the exchange argument is the specific tool that certifies a greedy strategy is not just fast, but actually optimal.*

Part II

Linear Data Structures

CHAPTER 7

Arrays: The Mathematics of Contiguous Memory

7.1 An Array Is a Linear Function

DEFINITION Array Addressing Formula

Memory is a giant one-dimensional sequence of addressable bytes. An array of n elements, each occupying s bytes, stored starting at base address B , places element at index i (zero-indexed) at address

$$\text{address}(i) = B + i \cdot s.$$

KEY INSIGHT

This one formula is the entire reason array access is $\Theta(1)$: computing $\text{address}(i)$ requires exactly one multiplication and one addition, regardless of n or i . Contrast this with a linked list (Chapter 8), where reaching index i requires i pointer-following steps because there is no closed-form address function — the address of node i depends on where the allocator happened to place node $i - 1$, which is not knowable without traversal.

WORKED EXAMPLE Computing an Address by Hand

An array of 32-bit integers ($s = 4$ bytes) begins at base address $B = 1000$. The address of index 7 is

$$\text{address}(7) = 1000 + 7 \times 4 = 1028.$$

No search, no traversal — one arithmetic computation regardless of whether the array holds 10 or 10 million elements.

7.2 Two-Dimensional Arrays: Row-Major Addressing

STEP-BY-STEP DERIVATION Address Formula for a 2D Array

A 2D array with R rows and C columns, stored in *row-major order* (each row laid out contiguously, one after another), places element (r, c) at

$$\text{address}(r, c) = B + (r \cdot C + c) \cdot s.$$

Why? To reach row r , you must skip past r complete rows, each of length C , contributing $r \cdot C$ elements; then move c more elements within the target row.

WORKED EXAMPLE A 3×4 Grid

A 3-row, 4-column array of 1-byte elements starting at $B = 0$. Element $(2, 1)$ (third row, second column, zero-indexed) sits at

$$\text{address}(2, 1) = 0 + (2 \times 4 + 1) \times 1 = 9.$$

7.3 The Cost of Insertion: A Shifting Argument

STEP-BY-STEP DERIVATION Why Inserting at Position k Costs $\Theta(n - k)$

To insert a new element at index k in an array of n elements while preserving contiguity, every element from index k to $n - 1$ must shift one position to the right, an operation performed $n - k$ times. Summed across all possible insertion points (assuming each is equally likely), the average shift cost is

$$\frac{1}{n+1} \sum_{k=0}^n (n-k) = \frac{1}{n+1} \cdot \frac{n(n+1)}{2} = \frac{n}{2} = \Theta(n).$$

KEY INSIGHT

Arrays trade insertion/deletion cost for lookup speed. This is the first concrete instance of the invariant-versus-cost trade-off introduced in Chapter 1: the invariant “elements are stored contiguously with no gaps” is exactly what makes the address formula work in $\Theta(1)$, but maintaining that invari-

ant after a middle insertion or deletion forces an $\Theta(n)$ shift.

7.4 Dynamic Arrays and the Doubling Strategy

A fixed-size array cannot grow. A *dynamic array* (as used to implement resizable lists) solves this by allocating a new, larger block and copying elements over when full. The mathematical question is: how much larger should the new block be?

STEP-BY-STEP DERIVATION Why Doubling Gives $\Theta(1)$ Amortized Insertion

Suppose the array doubles in capacity every time it fills up, starting from capacity 1. To perform n insertions, resizes happen when the array holds 1, 2, 4, 8, ... elements. The total cost of all the *copying* done across all resizes is

$$1 + 2 + 4 + \dots + 2^{\lceil \log_2 n \rceil} < 2n$$

by the geometric series formula from Chapter 3. Spread over n insertions, the average (*amortized*) copying cost per insertion is $\frac{2n}{n} = 2 = \Theta(1)$.

Contrast this with growing by a *fixed increment* c (e.g., always add room for 10 more): resizes then happen every c insertions, and the k -th resize copies $\approx kc$ elements, so total copying is $\sum_{k=1}^{n/c} kc = \Theta(n^2/c)$ — quadratic, and therefore $\Theta(n/c)$ amortized per insertion, which grows unboundedly as n grows. Doubling (geometric growth) is what makes the amortized cost a constant instead of a growing function of n .

KEY INSIGHT

This is your first taste of **amortized analysis** (formalized fully in Chapter 23): a single operation can occasionally be expensive ($\Theta(n)$ during a resize), yet the *average* cost per operation across a long sequence can still be $\Theta(1)$, provided the expensive operations become proportionally rarer — which is exactly what geometric (doubling) growth guarantees and arithmetic growth does not.

TAKEAWAY: An array's speed comes from a closed-form address function, $B + i \cdot s$, computable in constant time — and every array-based design decision (fixed size, doubling growth, shifting

cost) is a direct mathematical consequence of preserving that one formula.

CHAPTER 8

Linked Lists: The Mathematics of Pointer Chains

8.1 No Formula, Only a Chain

DEFINITION Linked Structure

A linked list stores n elements as n separately-allocated **nodes**, where node i stores a value and a pointer (reference) to node $i + 1$. There is no base-address formula as in Chapter 7; the only way to reach node i is to follow i pointers starting from the head.

STEP-BY-STEP DERIVATION Why Access Is $\Theta(n)$ but Insertion-at-a-Known-Point Is $\Theta(1)$

To reach index i , we must perform exactly i pointer dereferences: $\text{next}(\text{next}(\dots \text{next}(\text{head}) \dots))$, i times. Hence access/search is $\Theta(n)$ in the worst case (averaging $n/2$ steps).

But once we *already hold a pointer* to the node before the insertion point, inserting a new node is just two pointer reassignments:

$$\text{newNode.next} \leftarrow \text{prev.next}, \quad \text{prev.next} \leftarrow \text{newNode}.$$

This is $\Theta(1)$ regardless of n or the insertion position — no shifting of other elements is required, unlike the array case from Chapter 7.

KEY INSIGHT

This is the mirror image of the array's trade-off. Arrays offer $\Theta(1)$ access but $\Theta(n)$ middle insertion; linked lists offer $\Theta(n)$ access but $\Theta(1)$ insertion *given a pointer to the location*. Neither data structure is objectively “better” — the correct choice is a mathematical function of your workload's read/write ratio and access pattern, which is exactly the kind of trade-off analysis this

book trains you to perform.

8.2 Counting Pointers: A Structural Argument

WORKED EXAMPLE Total Pointers in Singly vs. Doubly Linked Lists

A singly linked list of n nodes has exactly n pointers (one *next* per node, with the last pointing to null). A doubly linked list has $2n$ pointers (*next* and *prev* per node). This constant-factor-2 increase in space buys a valuable capability: $\Theta(1)$ removal of a node given only a pointer to *that node itself* (no need to also hold the predecessor, since *prev* gives it directly) — impossible to do in $\Theta(1)$ with a singly linked list, where finding the predecessor to update its *next* pointer requires an $\Theta(n)$ scan from the head.

8.3 The Mathematics of Cycle Detection

DEFINITION Floyd's Cycle Detection — The Two-Pointer Argument

Given a linked list that may contain a cycle, use two pointers: a *slow* pointer that advances one node per step, and a *fast* pointer that advances two nodes per step. If a cycle exists, the two pointers are guaranteed to meet.

STEP-BY-STEP DERIVATION Why the Fast and Slow Pointers Must Meet

Model positions as integers modulo the cycle length c once both pointers have entered the cycle. At each step, the *gap* between the fast and slow pointer's position within the cycle increases by exactly 1 (since fast moves 2, slow moves 1, net gap change $+1$), modulo c . The gap therefore takes every value $0, 1, 2, \dots, c-1, 0, 1, \dots$ in sequence — a simple counting argument on $\mathbb{Z}/c\mathbb{Z}$. Since the gap is $0 \pmod{c}$ at the start of the process and increases by 1 each step, it must return to $0 \pmod{c}$ within at most c further steps — meaning the two pointers occupy the same node. This guarantees meeting within $O(n)$ total steps, using only $O(1)$ extra memory (just two pointers), which is the entire appeal of the technique versus a hash-set-based cycle check that would need $O(n)$ extra memory.

KEY INSIGHT

Notice the technique used here: reasoning about pointer positions as elements of *modular arithmetic* ($\mathbb{Z}/c\mathbb{Z}$) turns an apparently pointer-chasing problem into a clean number-theory argument. This “model the state as an integer mod something” trick recurs constantly in algorithm design — you will see it again in circular queues (Chapter 9) and open-addressing hash tables (Chapter 15).

TAKEAWAY: *A linked list has no address formula — only a chain of local relationships — which is precisely what makes pointer-based insertion $\Theta(1)$ but index-based access $\Theta(n)$; the entire structure is a direct trade of “no closed-form addressing” for “no shifting cost.”*

CHAPTER 9

Stacks and Queues: Order as a Mathematical Constraint

9.1 Stacks: Last-In-First-Out as a Formal Rule

DEFINITION Stack — LIFO

A stack supports push and pop, obeying the rule that the element removed is always the one most recently inserted. Formally, if $e_1, e_2, \dots, e_k$ are pushed in that order with no intervening pops, they must be popped in the exact reverse order $e_k, e_{k-1}, \dots, e_1$.

WORKED EXAMPLE Counting Valid Push/Pop Sequences — Catalan Numbers

Given n pushes and n pops interleaved in some order, how many interleavings are valid (i.e., never pop from an empty stack)? This is precisely the definition of the n -th **Catalan number**:

$$C_n = \frac{1}{n+1} \binom{2n}{n}.$$

For $n = 3$: $C_3 = \frac{1}{4} \binom{6}{3} = \frac{20}{4} = 5$ valid sequences. This same number C_n counts the number of distinct binary trees with n nodes, and the number of valid arrangements of n pairs of balanced parentheses — a beautiful structural coincidence, because a valid push/pop sequence, a balanced parenthesis string, and a binary tree shape are three different representations of the exact same combinatorial object.

KEY INSIGHT

Every operation on a stack — push, pop, peek — is $\Theta(1)$ because a stack only ever touches *one end* of the underlying storage (an array or linked list). There is no shifting (as in array-middle-insertion) and no traversal (as in linked-

list-middle-access) because the invariant “only touch the top” sidesteps both costs entirely.

9.2 Queues: First-In-First-Out and Modular Arithmetic

DEFINITION Queue — FIFO

A queue supports enqueue (insert at the back) and dequeue (remove from the front), preserving arrival order: the first element inserted is always the first removed.

9.2.1 The Circular Buffer: Modular Arithmetic in Action

A naive array-based queue wastes space, since dequeuing from the front either requires shifting all elements ($\Theta(n)$, defeating the purpose) or leaves a permanently growing gap at the front. The fix is a **circular buffer**.

STEP-BY-STEP DERIVATION Index Arithmetic for a Circular Queue

Let the underlying array have fixed capacity m . Maintain a `front` index and a `count` of elements currently stored. The position of the back slot (the next free slot to enqueue into) is

$$\text{back} = (\text{front} + \text{count}) \bmod m.$$

After an enqueue, $\text{count} \leftarrow \text{count} + 1$. After a dequeue, $\text{front} \leftarrow (\text{front} + 1) \bmod m$ and $\text{count} \leftarrow \text{count} - 1$. The modulo operation is what allows the index to “wrap around” from $m - 1$ back to 0, reusing freed slots at the front without ever shifting a single element. Every operation remains $\Theta(1)$.

WORKED EXAMPLE Tracing a Circular Queue of Capacity 5

Start empty, $\text{front} = 0, \text{count} = 0$. Enqueue A, B, C, D, E fills slots 0, 1, 2, 3, 4. Dequeue twice removes A, B : now $\text{front} = 2, \text{count} = 3$. Enqueue F : $\text{back} = (2 + 3) \bmod 5 = 0$, so F is placed at slot 0 — physically “before” C in the raw array, but logically after E in the queue’s order. This wraparound is exactly why we needed the modulo formula rather than raw indices.

9.3 Deques and the Symmetry of Two-Ended Access

DEFINITION Double-Ended Queue

A deque generalizes both stack and queue, supporting $\Theta(1)$ insertion and removal at *both* ends. Mathematically, it is a circular buffer (as above) where both *front* and *back* indices can move in either direction modulo m , e.g. $\text{front} \leftarrow (\text{front} - 1) \bmod m$ for a push to the front.

9.4 Stacks and Queues as Building Blocks

KEY INSIGHT

Stacks and queues are not just standalone tools — they are the control structures underneath two of the most important traversal strategies in this entire book. Depth-first search (Chapter 16) is nothing more than tree/graph traversal using a stack (explicit, or implicit via recursion) to always explore the most recently discovered node first. Breadth-first search is the exact same traversal skeleton, but using a queue to always explore the earliest discovered node first. The *only* difference between DFS and BFS, mathematically, is which of these two order-constraints (LIFO vs. FIFO) governs the frontier of nodes waiting to be explored.

TAKEAWAY: *Stacks and queues are order constraints, not data storage tricks — LIFO gives $\Theta(1)$ operations by only ever touching one end, FIFO does the same using modular arithmetic to wrap indices, and the choice between them is precisely what separates depth-first from breadth-first exploration everywhere else in this book.*

Part III

Trees

CHAPTER 10

Tree Foundations: Height, Depth, and Counting Nodes

10.1 A Tree Is a Connected, Acyclic Graph

DEFINITION Tree

A tree is a set of n nodes connected by exactly $n - 1$ edges, with a unique path between any two nodes (equivalently: connected, and containing no cycles). One node is designated the **root**; every other node has exactly one **parent**, and zero or more **children**.

STEP-BY-STEP DERIVATION Why a Tree on n Nodes Has Exactly $n - 1$ Edges

Build the tree by adding nodes one at a time, starting from a single root node (1 node, 0 edges). Every subsequent node, to remain connected without creating a cycle, must attach via *exactly one* new edge to the existing structure. After adding the remaining $n - 1$ nodes this way, we have added exactly $n - 1$ edges. Adding any node with 0 edges would disconnect it; adding any node with ≥ 2 edges to the existing tree would create a cycle. Hence $n - 1$ is not just typical but *forced* by the definition of a tree.

10.2 Depth, Height, and Levels

DEFINITION Depth, Height, Level

The **depth** of a node is the number of edges on the path from the root to that node (root has depth 0). The **height** of a tree is the maximum depth of any node (equivalently, the number of edges on the longest root-to-leaf path). **Level** k refers to the set of all nodes at depth k .

10.3 Counting Nodes in a Complete Binary Tree

STEP-BY-STEP DERIVATION Maximum Nodes at Height h

A binary tree has at most 2 children per node. Level 0 (the root) has at most $2^0 = 1$ node. Level 1 has at most $2^1 = 2$ nodes. Level k has at most 2^k nodes, since each of the (at most) 2^{k-1} nodes at level $k-1$ contributes at most 2 children. Summing across all levels from 0 to h using the geometric series (Chapter 3):

$$n_{\max} = \sum_{k=0}^h 2^k = 2^{h+1} - 1.$$

STEP-BY-STEP DERIVATION Inverting the Formula: Minimum Height for n Nodes

From $n \leq 2^{h+1} - 1$, solve for h :

$$n + 1 \leq 2^{h+1} \implies \log_2(n + 1) \leq h + 1 \implies h \geq \log_2(n + 1) - 1 = \Theta(\log n).$$

This proves that *any* binary tree storing n nodes must have height $\Omega(\log n)$ — you cannot pack n nodes into a shallower binary tree no matter how you arrange them. This is the fundamental lower bound that motivates balanced trees (Chapter 12): if we can *guarantee* a tree stays within a constant factor of this minimum height, every root-to-leaf operation costs only $O(\log n)$.

WORKED EXAMPLE Height Bound in Practice

A complete binary tree storing $n = 1,000,000$ nodes has height at least $\log_2(1,000,001) - 1 \approx 18.9$, so at least 19. Compare this to a completely unbalanced tree (effectively a linked list) storing the same million nodes: height 999,999. The gap between these two numbers — 19 versus a million — is the entire economic case for balanced trees, explored fully in Chapter 12.

10.4 Traversal Orders as Recursive Definitions

DEFINITION Depth-First Traversal Orders

Given a binary tree node with left subtree L and right subtree R :

- **Preorder:** visit node, then traverse L , then traverse R .
- **Inorder:** traverse L , then visit node, then traverse R .
- **Postorder:** traverse L , then traverse R , then visit node.

KEY INSIGHT

Each traversal order is a direct translation of the recursive structure of the tree itself into a sequence — this is the same induction-mirrors-recursion idea from Chapter 6, applied to trees. Inorder traversal of a binary *search* tree (Chapter 11) is especially important: it always visits nodes in sorted order, a direct consequence of the BST invariant ($\text{left} < \text{node} < \text{right}$) applied recursively at every level.

10.5 Counting All Possible Tree Shapes

WORKED EXAMPLE How Many Distinct Binary Trees on n Nodes?

As noted in Chapter 9, the number of distinct (unlabeled shape) binary trees with n nodes is the n -th Catalan number $C_n = \frac{1}{n+1} \binom{2n}{n}$, which grows as $\Theta(4^n / n^{1.5})$ — exponentially many shapes. This enormous space of possible shapes is exactly why we cannot simply “try all tree shapes” to find a good one; we need constructive invariants (like the balance conditions in Chapter 12) that *guarantee* a good shape by construction, rather than searching for one.

TAKEAWAY: *A tree’s power comes from the logarithmic relationship between height and node count, $h = \Omega(\log n)$ — but this bound is a ceiling on what’s possible, not a guarantee; the entire subject of tree balancing (Chapter 12) exists to force real trees to actually achieve it.*

CHAPTER 11

Binary Search Trees: An Invariant That Enables Search

11.1 The BST Invariant

DEFINITION Binary Search Tree Property

For every node x in the tree: every key in x 's left subtree is strictly less than x 's key, and every key in x 's right subtree is strictly greater. This invariant holds *recursively* at every node, not just at the root.

KEY INSIGHT

This single invariant is what turns a tree into a search structure. At any node, comparing the target key to the current node's key tells you, with certainty, which entire subtree to discard — the other subtree cannot possibly contain the target. This is structurally identical to the halving argument behind binary search on arrays (Chapter 4); a BST is binary search generalized to a shape that also supports cheap insertion and deletion.

11.2 Why Search, Insert, and Delete Are All $O(h)$

STEP-BY-STEP DERIVATION Search Cost Equals Path Length

Searching for key k starts at the root and, at each node, moves to the left or right child based on one comparison, terminating either at a match or at an empty subtree. The number of comparisons equals the number of nodes on the path from root to the terminating point — at most the height h of the tree. Hence search is $O(h)$. Insertion is search-plus-one-attachment, so also $O(h)$. Deletion is search-plus-a-bounded-amount-of-restructuring, also $O(h)$.

KEY INSIGHT

Combining this with Chapter 10's result that $h = \Omega(\log n)$ always, and h can be as bad as $\Theta(n)$ for a degenerate (e.g., sorted-order-inserted) tree, we get the central fact of this chapter: **a BST's performance is entirely a function of its height, and nothing about the basic BST definition controls that height.** This is precisely the gap that balanced trees (Chapter 12) close.

11.3 Average Height of a Randomly Built BST

STEP-BY-STEP DERIVATION Why Random Insertion Order Gives $O(\log n)$ Expected Height

Consider inserting n distinct keys into an initially empty BST in a *uniformly random order*. The key inserted first becomes the root; by symmetry, it is equally likely to be the 1st, 2nd, $\dots$, or n th smallest of the n keys. If it is the i th smallest, the left subtree ends up with $i - 1$ keys and the right subtree with $n - i$ keys, each subtree itself built by a uniformly random insertion order (a self-similar sub-problem). This gives a recurrence for the expected search-path length $D(n)$:

$$D(n) = 1 + \frac{1}{n} \sum_{i=1}^n [D(i-1) + D(n-i)].$$

This recurrence (closely related to the one for randomized quicksort's comparison count in Chapter 5) solves to $D(n) = \Theta(\log n)$. So while a *single* bad insertion order can create a $\Theta(n)$ -height tree, a *random* insertion order gives an expected $\Theta(\log n)$ height — mirroring exactly the average-versus-worst-case gap we saw for quicksort.

WORKED EXAMPLE Building a BST by Hand

Insert keys 5, 3, 8, 1, 4, 7, 9 in that order into an empty BST. 5 becomes the root. $3 < 5$ goes left; $8 > 5$ goes right. $1 < 5, 1 < 3$ goes to the left of 3. $4 < 5, 4 > 3$ goes to the right of 3. $7 > 5, 7 < 8$ goes to the left of 8. $9 > 5, 9 > 8$ goes to the right of 8. The resulting tree has height 2 (root at depth 0; 3, 8 at depth 1; 1, 4, 7, 9 at depth 2) — matching the $\Theta(\log n)$

prediction almost exactly, because this insertion order happens to be well-balanced.

11.4 Order Statistics: Augmenting the Invariant

DEFINITION Augmented BST for Rank Queries

If each node additionally stores the size of its own subtree ($\text{size}(x) = 1 + \text{size}(x.\text{left}) + \text{size}(x.\text{right})$), you can answer “what is the k -th smallest key?” in $O(h)$ time: at node x , compare k to $\text{size}(x.\text{left})$ to decide whether the answer lies in the left subtree, is x itself, or lies in the right subtree (recurring with an adjusted rank in the last case).

KEY INSIGHT

This is a first example of a recurring and powerful idea: **augmenting an invariant**. You can attach extra derived information to a data structure’s nodes (here, subtree size) as long as you can (a) compute it incrementally in $O(1)$ extra work per node during normal operations, and (b) use it to answer a new class of query efficiently. The same augmentation idea reappears when building interval trees, order-statistic trees, and many specialized structures beyond the scope of this book — but the pattern you should internalize is always the same two questions: what do I store, and can I maintain it for free alongside the operations I already have?

TAKEAWAY: *A BST’s cost is entirely governed by its height h — search, insert, and delete are all $O(h)$ — and while a random insertion order gives $\mathbb{E}[h] = \Theta(\log n)$, only an explicit balancing invariant (Chapter 12) can guarantee it against adversarial insertion orders.*

CHAPTER 12

Balanced Trees: Forcing Logarithmic Height

12.1 The Balance Invariant

DEFINITION AVL Balance Condition

An AVL tree is a BST with the extra invariant: for every node x , the heights of its left and right subtrees differ by at most 1:

$$| \text{height}(x.\text{left}) - \text{height}(x.\text{right}) | \leq 1.$$

This is checked and restored (via **rotations**) after every insertion and deletion.

12.2 Proving AVL Height Is $\Theta(\log n)$ — The Fibonacci Connection

This is one of the most beautiful derivations in classical data structure theory: we prove a *worst-case* logarithmic height guarantee, not just an average-case one.

STEP-BY-STEP DERIVATION Minimum Nodes in an AVL Tree of Height h

Let $N(h)$ be the *minimum* number of nodes an AVL tree of height h can have (the “worst,” most sparsely populated, but still-valid AVL shape). To minimize nodes at height h , one subtree should have height $h - 1$ (as small as allowed while still reaching height h) and, by the balance condition, the other subtree can have height as low as $h - 2$ — any lower would violate the ± 1 balance rule. So the minimum-node recurrence is

$$N(h) = 1 + N(h - 1) + N(h - 2), \quad N(0) = 1, N(-1) = 0.$$

Compare this to the Fibonacci recurrence $F(h) = F(h-1) + F(h-2)$ — $N(h)$ is (up to a shift and an added constant) literally a Fibonacci sequence. It is a standard fact that Fibonacci numbers grow exponentially, $F(h) = \Theta(\phi^h)$ where $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$ is the golden ratio. So

$$N(h) = \Theta(\phi^h).$$

STEP-BY-STEP DERIVATION Inverting to Bound the Height

Since a real tree of height h has *at least* $N(h) = \Theta(\phi^h)$ nodes, a tree with n nodes cannot have height larger than the h at which $N(h)$ first reaches n :

$$n \geq \Theta(\phi^h) \implies h \leq \Theta(\log_\phi n) = \Theta(\log n).$$

Combined with Chapter 10's universal lower bound $h = \Omega(\log n)$ for *any* binary tree, we get the guarantee we wanted:

$$h = \Theta(\log n) \quad \text{always, in the worst case, for any AVL tree.}$$

KEY INSIGHT

This is qualitatively different from the BST result in Chapter 11. There, we only proved $\mathbb{E}[h] = \Theta(\log n)$ under a random insertion order — an adversary choosing insertion order could still force $\Theta(n)$ height. Here, the balance invariant makes $h = \Theta(\log n)$ true *no matter what order* insertions and deletions arrive in, because the invariant is actively re-enforced after every single operation via rotations, rather than hoped for as a side effect of randomness.

12.3 Rotations: Restoring the Invariant in $O(1)$

DEFINITION Rotation

A rotation is a local restructuring around one node that changes the tree's shape (and hence subtree heights) while *preserving the BST invariant* (in-order sequence of keys is unchanged). A single rotation touches a constant number of pointers, so it costs $O(1)$.

STEP-BY-STEP DERIVATION Why Rebalancing After Insertion Costs $O(\log n)$ Total

Insertion first walks down the tree to find the insertion point ($O(h) = O(\log n)$ comparisons), then walks back up along the same path, performing at most $O(1)$ rotations at each of the $O(\log n)$ ancestor nodes to restore the balance invariant. Total cost: $O(\log n)$ for the search, plus $O(\log n)$ nodes visited during rebalancing, each doing $O(1)$ rotation work, giving $O(\log n)$ overall — the same asymptotic cost as an unbalanced BST's best case, but now *guaranteed*.

12.4 Red-Black Trees: A Looser, Cheaper Invariant

DEFINITION Red-Black Invariants

Each node is colored red or black, subject to: (1) the root is black; (2) red nodes never have red children; (3) every root-to-null path passes through the same number of black nodes (the tree's **black-height** bh).

STEP-BY-STEP DERIVATION Why Red-Black Height Is $O(\log n)$

Rule (2) forbids two consecutive red nodes on any path, so at most half the nodes on any root-to-leaf path can be red; hence any path has length at most $2 \cdot bh$. Separately, a subtree rooted where the black-height is bh has at least $2^{bh} - 1$ black-only nodes (by the same counting argument as Chapter 10's complete-tree bound, applied to the black nodes alone), giving $n \geq 2^{bh} - 1$, so $bh = O(\log n)$. Combining, $h \leq 2 \cdot bh = O(\log n)$.

KEY INSIGHT

Red-black trees permit a looser invariant than AVL (height can be up to roughly $2 \log n$ instead of AVL's tighter $\approx 1.44 \log n$), which means red-black trees need *fewer* rotations on average to restore their invariant after an update. This is exactly the same trade-off idea from Chapter 1 in a new guise: a stricter invariant (AVL) gives faster lookups (shorter trees) but costlier updates (more frequent rebalancing); a looser invariant (red-black) gives marginally slower lookups but cheaper updates — which is why red-black

trees are the more common choice in language standard libraries that favor balanced insert/lookup workloads.

TAKEAWAY: *Balance invariants convert the possibility of logarithmic height (Chapter 10) into a guarantee — AVL's tight ± 1 rule provably forces Fibonacci-slow growth in minimum node count, and every balanced tree design is a choice of how strict an invariant to enforce, trading lookup speed against rebalancing cost.*

CHAPTER 13

Heaps: A Weaker Invariant, Array-Packed

13.1 The Heap Property

DEFINITION Min-Heap Property

A binary min-heap is a complete binary tree (every level full except possibly the last, which fills left-to-right) where every node's key is $\leq$ both its children's keys. Note this is strictly weaker than the BST invariant — there is no ordering constraint between siblings or across subtrees, only along parent-child edges.

KEY INSIGHT

This weaker invariant is the whole point. A BST guarantees a total order is recoverable (via inorder traversal), which costs $O(\log n)$ per insertion to maintain. A heap only guarantees that the *minimum* is always easy to find, which turns out to require maintaining far less structure — and, remarkably, allows the entire tree to be packed into a flat array with no pointers at all.

13.2 The Array-Index Formula for a Complete Tree

STEP-BY-STEP DERIVATION Parent/Child Index Formulas

Because a heap is always a *complete* tree, we can number nodes $0, 1, 2, \dots$ in level order and derive index relationships with pure arithmetic — no pointers needed. For a node at index i (zero-indexed):

$$\text{parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor, \quad \text{leftChild}(i) = 2i+1, \quad \text{rightChild}(i) = 2i+2.$$

This follows directly from the geometric structure established in Chapter 10: level k begins at index $2^k - 1$, and each node at level k deterministically

produces two children at level $k + 1$ at the next two available slots.

WORKED EXAMPLE Verifying the Formula

Node at index 3: $\text{parent} = \lfloor (3 - 1)/2 \rfloor = 1$; children at $2(3) + 1 = 7$ and $2(3) + 2 = 8$. Node at index 0 (the root): $\text{parent} = \lfloor -1/2 \rfloor$, undefined as expected (root has no parent); children at 1 and 2.

KEY INSIGHT

This is a second instance of an addressing formula, echoing the array chapter (Chapter 7) — the heap achieves $\Theta(1)$ navigation between parent and child using pure arithmetic, exactly as arrays achieve $\Theta(1)$ indexed access, and for the same underlying reason: a rigid, predictable structure (complete tree shape, contiguous array) makes position computable instead of requiring stored pointers.

13.3 Sift-Up and Sift-Down: Cost Analysis

STEP-BY-STEP DERIVATION Why Insertion and Extraction Are $O(\log n)$

Insertion: place the new element at the next free array slot (preserving completeness), then repeatedly compare with its parent and swap upward (“sift-up”) while the heap property is violated. Each swap moves the element up one level; since the tree has height $\Theta(\log n)$ (a complete tree with n nodes, by Chapter 10’s bound applied with equality since complete trees are maximally packed), at most $O(\log n)$ swaps occur.

Extract-min: remove the root, move the last array element to the root position (preserving completeness), then repeatedly compare with children and swap downward (“sift-down”) while violated. Again bounded by the tree’s height, $O(\log n)$.

13.4 Building a Heap from Scratch: Why It's $\Theta(n)$, Not $\Theta(n \log n)$

This is a classic case where a naive upper-bound estimate is loose, and a more careful summation (echoing Chapter 3's techniques) reveals the true, better bound.

STEP-BY-STEP DERIVATION Tight Analysis of Build-Heap

The standard BUILD-HEAP procedure calls sift-down on every internal node, working from the bottom of the tree upward. A node at height k above the leaves (i.e., a subtree of height k rooted there) costs $O(k)$ for its sift-down — not $O(\log n)$, because the element only ever needs to sift down as far as that particular subtree's height, which shrinks the closer the node is to the bottom.

There are at most $\lceil n/2^{k+1} \rceil$ nodes at height k (Chapter 10's level-counting argument). Total work:

$$\sum_{k=0}^{\log n} \frac{n}{2^{k+1}} \cdot O(k) = O\left(n \sum_{k=0}^{\infty} \frac{k}{2^{k+1}}\right).$$

The sum $\sum_{k=0}^{\infty} k/2^{k+1}$ is a standard convergent arithmetico-geometric series equal to a constant ($= 1$, in fact). So the total cost is

$$O(n \cdot 1) = O(n).$$

KEY INSIGHT

This result — that building a heap from n unordered elements takes only $\Theta(n)$, not $\Theta(n \log n)$ — is a direct consequence of the fact that *most nodes in a tree are near the leaves* (recall from Chapter 3 that half of all nodes are leaves, a quarter are one level up, and so on), and near-leaf nodes have very little distance left to sift down. Whenever you see a bound derived by simply multiplying (number of nodes) by (worst-case cost per node), ask whether the true per-node cost actually varies across nodes — a weighted sum, like this one, is frequently tighter than a uniform worst-case bound.

13.5 Heap Sort: Reusing the Heap Invariant

STEP-BY-STEP DERIVATION Heap Sort's Total Cost

Build a max-heap in $\Theta(n)$ (as derived above). Then repeatedly extract the maximum (swap it to the end of the array, shrink the heap by one, sift-down the new root) — each of the n extractions costs $O(\log n)$. Total:

$$\Theta(n) + n \cdot O(\log n) = O(n \log n).$$

This matches the comparison-sort lower bound proved in Chapter 5 via counting arguments — heap sort is therefore asymptotically optimal among comparison sorts, and unlike merge sort (Chapter 4), it achieves this *in place*, using $O(1)$ extra memory, precisely because the heap invariant lets it reuse the input array itself as the implicit tree structure.

TAKEAWAY: *A heap trades the BST's strong total-order invariant for a weaker parent-child-only invariant, and that weaker invariant is exactly what allows $\Theta(1)$ arithmetic navigation (no pointers), $\Theta(n)$ construction via a weighted per-level cost analysis, and an optimal $\Theta(n \log n)$, in-place sort.*

CHAPTER 14

Tries: Trading Key Length for Key Count

14.1 An Invariant Based on Shared Prefixes

DEFINITION Trie

A trie (prefix tree) stores a set of strings by representing each string as a root-to-node path, where each edge is labeled with one character, and strings sharing a common prefix share the corresponding path. A node is marked “terminal” if the path to it spells a complete stored string.

KEY INSIGHT

The key structural insight is what the trie’s cost depends on: unlike a BST, whose cost is a function of n (the number of stored keys) via its height $\Theta(\log n)$, a trie’s search cost is a function of L , the *length of the query string* — completely independent of how many strings n are stored. This is a fundamentally different trade-off axis from every structure so far in this book.

14.2 Cost Analysis

STEP-BY-STEP DERIVATION Why Search, Insert, and Delete Are All $\Theta(L)$

Searching for a string of length L follows exactly L edges, one character at a time, regardless of how many other strings share or don’t share those prefixes. Each step is $O(1)$ if child lookup is done via a fixed-size array indexed by character (alphabet size Σ), or $O(\log |\Sigma|)$ if done via a balanced structure per node. So:

$$T_{\text{search}}(L) = \Theta(L), \quad \text{independent of } n.$$

WORKED EXAMPLE Why This Beats a Hash Set for Prefix Queries

A hash set (Chapter 15) can test membership of one exact string in expected $O(L)$ (the time to hash the string), matching a trie’s cost for single lookups. But a hash set *cannot* efficiently answer “list all stored strings with prefix ‘pre’” — doing so requires scanning all n entries, $\Theta(n)$. A trie answers the same prefix query in $\Theta(L_{\text{prefix}} + k)$, where k is the number of matches: walk down the L_{prefix} characters of the prefix, then traverse the subtree rooted there, visiting exactly the k terminal nodes that match and nothing else. This is the mathematical reason autocomplete systems use tries (or their compressed cousins) rather than hash sets.

14.3 The Space–Time Trade: Counting Nodes

STEP-BY-STEP DERIVATION Worst-Case Node Count

Storing n strings with average length L using a naive trie creates at most $n \cdot L$ nodes (one new node per character, in the worst case that no prefixes are shared at all). Space is therefore $O(nL)$ in the worst case — potentially far worse than the $O(nL)$ needed just to *store* the strings themselves as raw text, since a trie adds per-node overhead (child pointers/arrays) on top. When many strings *do* share prefixes, shared paths reduce the actual node count well below nL , which is precisely the situation (e.g., dictionary words, URLs, IP-routing prefixes) where tries earn their keep.

KEY INSIGHT

A **compressed trie** (also called a radix tree or Patricia trie) merges any chain of single-child nodes into one edge labeled with a multi-character substring, reducing node count to $O(n)$ regardless of string length — a direct application of the same idea as run-length shortening: don’t pay per-character overhead where there is no branching decision to make.

TAKEAWAY: A trie replaces “cost depends on n ” with “cost depends on key length L ,” by directly encoding shared prefixes as shared paths — the right choice whenever your workload does prefix-based queries that a hash table’s flat, unordered bucketing cannot answer efficiently.

Part IV

Hashing

CHAPTER 15

Hashing: Turning Keys into Addresses

15.1 The Core Idea: Simulate an Array Formula for Arbitrary Keys

Recall from Chapter 7 that an array gives $\Theta(1)$ access via the formula $\text{address}(i) = B + i \cdot s$ — but only because indices are already small integers. Hashing’s entire goal is to build an analogous formula for keys that are *not* small integers (strings, objects, tuples): a **hash function** h that maps any key to a small integer index.

DEFINITION Hash Function

A hash function $h : U \rightarrow \{0, 1, \dots, m - 1\}$ maps keys from a (potentially huge) universe U to **slots** in a table of size m . Storage then uses $\text{address}(k) = B + h(k) \cdot s$ — identical in form to the array formula, with $h(k)$ playing the role of the index.

15.2 Why Collisions Are Mathematically Unavoidable

By the pigeonhole principle (Chapter 5), if $|U| > m$ (virtually always true — think of all possible strings versus a table of size 1,000), some two keys *must* map to the same slot no matter how h is designed. The engineering goal is therefore never “eliminate collisions” (impossible) but “make collisions rare and cheap to resolve.”

15.3 Designing a Hash Function: The Division and Multiplication Methods

DEFINITION Division Method

$$h(k) = k \bmod m.$$

Simple and fast, but sensitive to m 's structure: if $m = 2^r$, $h(k)$ depends only on the low r bits of k , ignoring the rest of the key entirely — a poor spread if low bits are non-random. Choosing m to be a prime not close to a power of 2 typically avoids this pathology.

DEFINITION Multiplication Method

$$h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor, \quad 0 < A < 1 \text{ (commonly } A \approx (\sqrt{5} - 1)/2\text{)}.$$

Here $kA \bmod 1$ denotes the fractional part of kA . This method is far less sensitive to the choice of m , since multiplying by an irrational-like constant spreads bit patterns across the whole key rather than isolating specific bits.

15.4 Load Factor: The Central Health Metric

DEFINITION Load Factor

$$\alpha = \frac{n}{m}$$

where n is the number of stored keys and m is the table size. α is the average number of keys per slot, and it is the single number that governs expected performance.

15.4.1 Chaining: Expected Cost as a Direct Function of α

STEP-BY-STEP DERIVATION Expected Search Cost Under Chaining

With **chaining** (each slot holds a linked list of all keys hashed there), assume **simple uniform hashing**: any key is equally likely to land in any of the m slots, independent of other keys. The expected number of keys in any one slot's chain is, by linearity of expectation (Chapter 5),

$$\mathbb{E}[\text{chain length}] = \sum_{\text{keys } k_i} \Pr[h(k_i) = \text{this slot}] = n \cdot \frac{1}{m} = \alpha.$$

An unsuccessful search must scan the entire chain: expected cost $\Theta(1 + \alpha)$ (the $+1$ for computing the hash itself). A successful search is on average about half that, still $\Theta(1 + \alpha)$. If $\alpha = O(1)$ — i.e., the table is resized to keep $n = O(m)$, exactly as with dynamic arrays in Chapter 7 — search is $\Theta(1)$ expected time.

15.4.2 Open Addressing: Probe Sequences

DEFINITION Open Addressing

Instead of chains, all keys live directly in the table. On a collision, a **probe sequence** $h(k, 0), h(k, 1), h(k, 2), \dots$ tries successive alternative slots until an empty one is found. **Linear probing**: $h(k, i) = (h(k) + i) \bmod m$. **Double hashing**: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$.

STEP-BY-STEP DERIVATION Expected Probes Under Uniform Hashing

Under the idealized assumption that each probe sequence is equally likely to be any of the $m!$ permutations of slots (uniform hashing), the expected number of probes for an unsuccessful search is

$$\mathbb{E}[\text{probes}] = \frac{1}{1 - \alpha}.$$

Derivation sketch: the probability the first probe hits an occupied slot is α (fraction full); given that, the probability the second also hits occupied is $\approx \frac{n-1}{m-1} \approx \alpha$; and so on, giving an expected count that sums a geometric-like

series $1 + \alpha + \alpha^2 + \dots = \frac{1}{1-\alpha}$ using the infinite geometric series formula from Chapter 3.

KEY INSIGHT

Look at what happens as $\alpha \rightarrow 1$ (table nearly full): $\frac{1}{1-\alpha} \rightarrow \infty$. This is why open addressing degrades sharply near full capacity, in a way chaining (which degrades only linearly, $\Theta(1 + \alpha)$) does not — and why open-addressed hash tables typically trigger a resize at a stricter threshold (e.g., $\alpha = 0.7$) than chained tables.

WORKED EXAMPLE Comparing $\alpha = 0.5$ vs. $\alpha = 0.9$ Under Open Addressing

At $\alpha = 0.5$: expected probes = $\frac{1}{1-0.5} = 2$. At $\alpha = 0.9$: expected probes = $\frac{1}{1-0.9} = 10$. A mere jump from half-full to 90% full quintuples the expected search cost — a direct, quantitative illustration of why hash tables resize well before becoming full.

15.5 Resizing: Reusing the Amortized Argument

KEY INSIGHT

Exactly as with dynamic arrays (Chapter 7), doubling the table size whenever α crosses a threshold gives $\Theta(1)$ *amortized* insertion cost — the same geometric-series argument applies verbatim, because the total cost of all re-hashings across n insertions is again bounded by a geometric series summing to $O(n)$.

15.6 Universal Hashing: Defeating Adversarial Inputs

DEFINITION Universal Hash Family

A family $\mathcal{H}$ of hash functions is **universal** if, for any two distinct keys $k_1 \neq k_2$, the fraction of functions $h \in \mathcal{H}$ for which $h(k_1) = h(k_2)$ is at most $1/m$:

$$\Pr_{h \in \mathcal{H}} [h(k_1) = h(k_2)] \leq \frac{1}{m}.$$

KEY INSIGHT

A single fixed hash function can always be defeated by an adversary who knows it, feeding in n keys engineered to all collide (reducing performance to $\Theta(n)$, as bad as a linked list — this is a real denial-of-service vector in naive implementations). Choosing h *randomly at runtime* from a universal family means no input can be chosen in advance to cause bad behavior, because the attacker cannot predict which specific h will be used. This restores the $\Theta(1 + \alpha)$ expected-time guarantee even against an adversary who knows your source code, and is exactly the birthday-paradox-style probability reasoning from Chapter 5 turned into a security property.

TAKEAWAY: Hashing reconstructs the array's $\Theta(1)$ addressing formula for arbitrary keys via $h(k)$; the pigeonhole principle makes collisions mathematically inevitable, so all of hash table design reduces to controlling the load factor α (via resizing) and choosing h well (via universal families), since expected cost is a direct, provable function of α alone.

Part V

Graphs

CHAPTER 16

Graph Foundations and Traversal

16.1 Graphs Generalize Trees

DEFINITION Graph

A graph $G = (V, E)$ consists of a set of $n = |V|$ vertices and $m = |E|$ edges, each edge connecting two vertices. Unlike a tree, a graph permits cycles and need not be connected. If edges have direction, G is a **directed graph** (digraph); otherwise **undirected**.

16.2 The Handshake Lemma

STEP-BY-STEP DERIVATION Sum of Degrees Equals $2m$

The **degree** of a vertex is its number of incident edges. Since every edge (u, v) contributes exactly 1 to u 's degree and 1 to v 's degree, summing degrees over all vertices counts every edge exactly twice:

$$\sum_{v \in V} \deg(v) = 2m.$$

KEY INSIGHT

This simple identity has an immediate algorithmic consequence: the *average* degree in any graph is $2m/n$, which means if a graph is **sparse** ($m = O(n)$), average degree is $O(1)$ — most algorithms that touch each vertex's neighbor list therefore do $O(1)$ work per vertex on average, a fact used constantly in complexity analysis of graph algorithms.

16.3 Two Representations, Two Cost Profiles

DEFINITION Adjacency Matrix vs. Adjacency List

An **adjacency matrix** is an $n \times n$ 0/1 matrix A where $A[i][j] = 1$ iff edge (i, j) exists. An **adjacency list** stores, for each vertex, a list of its neighbors.

WORKED EXAMPLE Space and Query Trade-offs

Adjacency matrix: $\Theta(n^2)$ space regardless of m ; edge existence query (i, j) in $\Theta(1)$ (direct array lookup, echoing Chapter 7); listing all neighbors of v costs $\Theta(n)$ (scan the whole row).

Adjacency list: $\Theta(n + m)$ space; edge existence query costs $O(\deg(v))$ (scan v 's list); listing all neighbors of v costs $\Theta(\deg(v))$ — optimal, since you must output exactly that many neighbors.

For sparse graphs ($m \ll n^2$, the overwhelmingly common case in practice — road networks, social graphs, web graphs), adjacency lists save enormous space: $\Theta(n + m)$ versus $\Theta(n^2)$ is a difference of orders of magnitude once n reaches the millions.

16.4 Breadth-First Search: Layered by Distance

DEFINITION BFS

Starting from source s , BFS explores vertices in order of increasing distance (number of edges) from s , using a queue (Chapter 9) to always expand the earliest-discovered, not-yet-explored vertex next.

STEP-BY-STEP DERIVATION Why BFS Runs in $\Theta(n + m)$

Each vertex is enqueued exactly once (guarded by a “visited” check), so $\Theta(n)$ total enqueue/dequeue operations. When a vertex is dequeued, we scan its adjacency list once, an operation whose cost across *all* dequeues sums, by definition of degree, to $\sum_v \deg(v) = \Theta(m)$ by the handshake lemma above. Total: $\Theta(n) + \Theta(m) = \Theta(n + m)$.

STEP-BY-STEP DERIVATION Why BFS Computes Shortest Paths in Unweighted Graphs

Claim: when BFS dequeues vertex v , its recorded distance $d(v)$ equals the true shortest-path distance from s .

Proof (induction on BFS layers, echoing Chapter 6): vertices are dequeued in non-decreasing order of $d(v)$ because the queue is FIFO and each new discovery sets distance to (parent's distance + 1). **Base case:** $d(s) = 0$, trivially correct. **Inductive step:** assume all vertices dequeued so far have correct shortest distances. Any vertex v discovered from a vertex u at distance $d(u)$ is assigned $d(v) = d(u) + 1$. If a shorter path to v existed via some other vertex u' , u' would have distance $< d(u)$, and since BFS processes vertices in non-decreasing distance order, u' would have been dequeued (and would have discovered v) before u was — contradiction. So $d(v) = d(u) + 1$ is indeed the shortest distance. ■

16.5 Depth-First Search: Stack Order and Edge Classification

DEFINITION DFS

Identical traversal skeleton to BFS, but using a stack (explicit, or implicit via recursion) instead of a queue — always expanding the *most recently* discovered vertex next.

KEY INSIGHT

DFS's cost is also $\Theta(n + m)$ by the identical argument as BFS (each vertex popped once, each adjacency list scanned once, summing to $\Theta(m)$ via the handshake lemma). The mathematical value of DFS comes from the extra structure the recursion (stack) reveals: each edge can be classified as a **tree edge**, **back edge** (to an ancestor — signals a cycle), **forward edge**, or **cross edge**, based on the relative discovery/finish times of its endpoints. **A graph has a cycle if and only if DFS finds a back edge** — a single, clean characterization that underlies cycle detection, topological sorting, and strongly-connected-component algorithms.

STEP-BY-STEP DERIVATION Topological Sort via DFS Finish Times

For a directed acyclic graph (DAG), running DFS and recording each vertex's **finish time** (when its recursive exploration completes) gives a valid topological order by listing vertices in *decreasing* finish time. **Why it works:** for any edge (u, v) , DFS must finish exploring v (and everything reachable from it) before it can finish exploring u , because v was discovered during u 's exploration and u 's finish time is only recorded after all its descendants' subtrees are fully explored. Hence $\text{finish}(v) < \text{finish}(u)$ for every edge $u \rightarrow v$, which is exactly the condition a topological order (all edges point from earlier to later in the order) requires when listed by decreasing finish time.

TAKEAWAY: Both BFS and DFS are $\Theta(n + m)$ because every vertex is processed once and every edge is scanned exactly twice (via the handshake lemma) — the only difference between them is a stack versus a queue governing traversal order, yet that one choice is the difference between shortest-path layers and cycle/ordering structure.

CHAPTER 17

Shortest Paths: Relaxation as a Mathematical Operation

17.1 The Relaxation Operation

DEFINITION Edge Relaxation

For an edge (u, v) with weight $w(u, v)$, maintaining a running best-known distance estimate $d(\cdot)$ from source s , **relaxing** the edge means:

$$\text{if } d(v) > d(u) + w(u, v) : \quad d(v) \leftarrow d(u) + w(u, v).$$

Every shortest-path algorithm in this chapter is, underneath, a different strategy for deciding *which order* to relax edges in.

KEY INSIGHT

Relaxation can only ever *decrease* an estimate, and only when it finds genuine evidence of a shorter path. This monotonicity is what makes correctness proofs for shortest-path algorithms tractable: $d(v)$ never gets worse, so once we can argue $d(v)$ has reached its true minimum value, it stays correct forever after.

17.2 Dijkstra's Algorithm: A Greedy Proof

DEFINITION Dijkstra's Strategy

Maintain a set S of vertices whose shortest distance is already finalized. Repeatedly select the vertex $u \notin S$ with the smallest tentative $d(u)$, add it to S , and relax all of u 's outgoing edges. Requires all edge weights to be non-negative.

STEP-BY-STEP DERIVATION Proof of Correctness via Exchange/Contradiction — Echoing Chapter 6

Claim: when a vertex u is added to S , $d(u)$ equals its true shortest distance from s .

Proof (induction on the order vertices are added to S , by contradiction): suppose u is the first vertex added to S with $d(u)$ *not* equal to the true shortest distance $\delta(u)$. Since $d(u) \geq \delta(u)$ always holds (relaxation only ever finds valid path lengths, never an under-estimate), we must have $d(u) > \delta(u)$, meaning some shorter true path P from s to u exists. Let y be the first vertex on P not yet in S , and x the vertex just before it on P (so $x \in S$, by minimality of u as the first mistake). Because all edge weights are non-negative, the prefix of P up to y has length $\leq \delta(u) < d(u)$. But $x \in S$ means x 's edges were already relaxed, so $d(y) \leq d(x) + w(x, y) = \delta(y) \leq \delta(u) < d(u)$. This contradicts the algorithm's choice of u as the vertex with *smallest* tentative d among vertices outside S (since y would have had a smaller value). Hence no such mistake u can exist. ■

Notice this proof *requires* non-negative weights — the step “the prefix of P up to y has length $\leq \delta(u)$ ” relies on the remaining portion of P (from y to u) contributing a non-negative amount, which fails with negative edges. This is precisely why Dijkstra's algorithm is unsound on graphs with negative weights.

STEP-BY-STEP DERIVATION Running Time via a Priority Queue

Selecting the minimum- d vertex n times, using a binary heap (Chapter 13) as a priority queue, costs $O(\log n)$ per extraction: $O(n \log n)$ total. Each of the m edges is relaxed at most once, and a relaxation that changes $d(v)$ requires a $O(\log n)$ priority queue update. Total:

$$T(n, m) = O((n + m) \log n).$$

For sparse graphs ($m = O(n)$), this is $O(n \log n)$ — a direct payoff of the heap's logarithmic operations from Chapter 13.

17.3 Bellman-Ford: Trading Speed for Generality

DEFINITION Bellman-Ford Strategy

Relax *every* edge in the graph, repeated for $n - 1$ rounds (no priority queue — just brute-force repetition).

STEP-BY-STEP DERIVATION Why $n - 1$ Rounds Suffice

Any shortest path in a graph with n vertices visits at most $n - 1$ edges (more would revisit a vertex, and since we assume no negative cycles, revisiting can never help). By induction on round number r : after r rounds of relaxing all edges, every shortest path using at most r edges has been correctly found. This is because each round extends the “correctly solved” path-length frontier by exactly one edge, mirroring the layer-by-layer argument used for BFS in Chapter 16, but for edge-count rather than weight. After $n - 1$ rounds, all shortest paths (which use at most $n - 1$ edges) are correctly computed. Total cost: $n - 1$ rounds $\times$ m edge relaxations per round $= O(nm)$.

KEY INSIGHT

Bellman-Ford is asymptotically slower than Dijkstra ($O(nm)$ vs. $O((n + m) \log n)$) but handles negative edge weights correctly, because its correctness proof never assumed non-negativity — it only assumed the absence of a negative-weight *cycle* (which would make “shortest path” undefined, since you could loop forever to decrease the total weight without bound). As a bonus, a graph has a negative cycle if and only if a distance still decreases on an n -th round of relaxation — a direct, cheap consequence of the induction argument above.

17.4 Floyd-Warshall: All-Pairs via Dynamic Programming

STEP-BY-STEP DERIVATION The Recurrence Behind Floyd-Warshall

Let $d_k(i, j)$ be the shortest distance from i to j using only intermediate vertices from $\{1, \dots, k\}$. The key recurrence (a preview of the dynamic-programming ideas formalized in Chapter 20):

$$d_k(i, j) = \min \left(d_{k-1}(i, j), d_{k-1}(i, k) + d_{k-1}(k, j) \right)$$

with base case $d_0(i, j) = w(i, j)$ (direct edge weight, or ∞ if none). The recurrence says: either the best path from i to j avoids vertex k entirely (first term), or it passes through k exactly once (second term) — since any path through k more than once could shortcut the repeat and only get better. Since k ranges over all n vertices, and (i, j) ranges over all n^2 pairs, filling this table costs $O(n^3)$ total, each cell computed in $O(1)$ from already-computed smaller- k entries.

KEY INSIGHT

Floyd-Warshall trades a worse asymptotic complexity ($O(n^3)$, versus running Dijkstra from every source at $O(n(n + m) \log n)$) for radical simplicity and, crucially, correctness with negative edges (still no negative cycles) with no priority queue machinery — a good illustration that “asymptotically faster” and “practically better” are not always the same choice, and part of algorithm design is knowing when a simpler $O(n^3)$ recurrence beats a cleverer but more complex $O(n^2 \log n)$ -ish alternative for the problem sizes you actually face.

TAKEAWAY: Every shortest-path algorithm is a strategy for ordering edge relaxations — Dijkstra’s greedy order is provably correct only for non-negative weights via a contradiction argument, Bellman-Ford’s brute-force $n - 1$ rounds sacrifice speed for handling negative weights, and Floyd-Warshall reframes the entire problem as a dynamic-programming recurrence over intermediate vertices.

CHAPTER 18

Minimum Spanning Trees: The Cut Property

18.1 The Problem, Formally

DEFINITION Minimum Spanning Tree

Given a connected, undirected graph $G = (V, E)$ with edge weights w , a **spanning tree** is a subset of $n - 1$ edges (recall Chapter 10: exactly $n - 1$ edges are needed and sufficient to connect n vertices acyclically) that connects all vertices. A **minimum** spanning tree (MST) minimizes the sum of edge weights among all spanning trees.

18.2 The Cut Property: The Master Theorem of MSTs

Almost every MST algorithm is a direct application of one theorem.

DEFINITION Cut Property

A **cut** is any partition of V into two non-empty sets $(A, V \setminus A)$. A **crossing edge** has one endpoint in each side. **Cut Property:** for any cut, the minimum-weight crossing edge belongs to *some* MST (assuming distinct edge weights, for simplicity).

STEP-BY-STEP DERIVATION Proof of the Cut Property, by Exchange Argument — Echoing Chapter 6

Let $e = (u, v)$ be the minimum-weight edge crossing a cut $(A, V \setminus A)$. Suppose, for contradiction, some MST T does not contain e . Since T is a spanning tree, adding e to T creates exactly one cycle (Chapter 10: a tree plus one edge always creates exactly one cycle). This cycle, since it must return from u 's side back to v 's side, contains at least one other cross-

ing edge e' of the same cut. Because e has minimum weight among all crossing edges, $w(e) \leq w(e')$. Form $T' = T - e' + e$: this is still a spanning tree (removing e' breaks the cycle we just created, restoring acyclicity, while connectivity is preserved because e still connects the two pieces), and $w(T') = w(T) - w(e') + w(e) \leq w(T)$. So T' is a spanning tree at least as good as T , and it contains e . Hence some MST contains e . ■

KEY INSIGHT

This is the exact same exchange-argument template used to prove greedy correctness in Chapter 6's activity-selection example: assume an optimal solution disagrees with the greedy choice, then exchange a piece to show the greedy choice could have been included without loss. **Every** MST algorithm below is just a different systematic way of choosing cuts and repeatedly invoking this one property.

18.3 Kruskal's Algorithm: Sort, Then Greedily Union

DEFINITION Kruskal's Strategy

Sort all m edges by weight. Process them in increasing order, adding an edge to the growing forest if and only if it connects two currently-separate components (i.e., does not create a cycle).

STEP-BY-STEP DERIVATION Correctness via the Cut Property

When Kruskal's algorithm considers edge $e = (u, v)$ and finds u, v in different components, consider the cut $(A, V \setminus A)$ where A is u 's current component. Every edge processed so far either lies within A or within $V \setminus A$ (else it would have already merged the two), so e is the *minimum-weight edge not yet excluded* crossing this cut — and since edges are processed in sorted order, e is in fact the minimum-weight crossing edge overall. By the cut property, including e is safe.

STEP-BY-STEP DERIVATION Running Time

Sorting m edges: $O(m \log m)$. Processing each edge requires a **union-find** (disjoint-set) structure to test “are u, v already connected?” and to merge components — with path compression and union-by-rank, this costs $O(\alpha(n))$ *amortized* per operation, where α is the inverse Ackermann function, growing so slowly it is ≤ 4 for any n realistically representable in a computer, making it effectively $O(1)$ in practice. Total: $O(m \log m)$, dominated by the sort. Since $m \log m = O(m \log n)$ (as $m \leq n^2 \Rightarrow \log m \leq 2 \log n$), this is usually written $O(m \log n)$.

18.4 Prim's Algorithm: Grow One Tree from a Seed

DEFINITION Prim's Strategy

Start from an arbitrary vertex, forming a single-vertex tree T . Repeatedly add the minimum-weight edge that connects a vertex in T to a vertex outside T , until all vertices are included.

STEP-BY-STEP DERIVATION Correctness via the Cut Property

At every step, the cut is $(T, V \setminus T)$ by construction. Prim's algorithm always adds the minimum-weight edge crossing exactly this cut, which the cut property certifies as safe to add. This makes Prim's correctness proof arguably even more direct than Kruskal's, since the cut is always explicitly “current tree vs. rest” rather than an implicit one revealed by sorted order.

KEY INSIGHT

Notice the structural resemblance between Prim's algorithm and Dijkstra's algorithm (Chapter 17): both grow a structure one vertex at a time, using a priority queue to always pick the cheapest next option. The difference is entirely in *what* is minimized — Dijkstra minimizes total distance-from-source $d(v)$, while Prim minimizes the single edge weight connecting v to the current tree. This is a beautiful example of how the same algorithmic skeleton (priority-queue-driven greedy growth) can solve two different problems depending on which quantity keys the priority queue.

STEP-BY-STEP DERIVATION Running Time with a Binary Heap

Same structure as Dijkstra: n extract-min operations at $O(\log n)$ each, and at most m decrease-key operations at $O(\log n)$ each (Chapter 13's heap operations). Total: $O((n + m) \log n)$, identical in form to Dijkstra's bound for the same reason — both are heap-driven greedy vertex-growth algorithms.

TAKEAWAY: *The cut property — that the minimum crossing edge of any cut belongs to some MST — is the single theorem underlying both Kruskal's (implicit cuts via sorted order) and Prim's (explicit tree-vs-rest cut) algorithms, and is itself proved by the same exchange-argument technique used to certify greedy algorithms throughout this book.*

Part VI

Algorithm Design Paradigms

CHAPTER 19

Divide and Conquer: Recursion as a Design Strategy

19.1 The Three-Step Template

DEFINITION Divide and Conquer

Every divide-and-conquer algorithm follows the same three-step template:

1. **Divide** the problem into a subproblems, each of size roughly n/b .
2. **Conquer** each subproblem recursively (base case: solve directly when small enough).
3. **Combine** the subproblem solutions into a solution for the original problem.

The Master Theorem (Chapter 4) is the general-purpose tool for analyzing any algorithm fitting this template, via the recurrence $T(n) = aT(n/b) + f(n)$, where $f(n)$ captures the cost of dividing and combining.

19.2 Merge Sort, Revisited as a Design Choice

We already analyzed merge sort's recurrence in Chapter 4. The design insight worth isolating here is *why* splitting in half is the right choice, not just an arbitrary one.

KEY INSIGHT

Splitting into $a = 2$ equal halves ($b = 2$) with $\Theta(n)$ combine cost puts us squarely in Master Theorem Case 2 ($f(n) = \Theta(n^{\log_2 2}) = \Theta(n)$), giving $\Theta(n \log n)$. Had we instead split into $n/1$ pieces of size 1 directly (i.e., no recursive structure — repeatedly merge one element at a time), the combine cost per level structure collapses and we'd effectively reproduce insertion sort's $\Theta(n^2)$. The *balance* of the split ($n/2$ and $n/2$, not $n - 1$ and 1) is what

keeps the recursion tree shallow ($\Theta(\log n)$ levels) — an unbalanced split, even though it still “divides and conquers,” produces a tall, skinny recursion tree with $\Theta(n)$ levels instead, exactly like the unbalanced BST from Chapter 11.

19.3 Quickselect: Divide and Conquer Without Combining

DEFINITION Quickselect

To find the k -th smallest element of an unsorted array: pick a pivot, partition the array around it (elements less than pivot on the left, greater on the right — $\Theta(n)$ work), then recurse into *only one side* (whichever side contains the k -th element), never both.

STEP-BY-STEP DERIVATION Why Expected Time Is $\Theta(n)$, Not $\Theta(n \log n)$

With a random pivot, the partition step splits the array into two parts whose sizes are, in expectation, a constant fraction of n each. The recurrence for expected cost is

$$T(n) = T(n/2) + \Theta(n) \quad (\text{informally, using expected split size}).$$

Apply the Master Theorem: $a = 1, b = 2, f(n) = \Theta(n)$. Watershed: $n^{\log_2 1} = n^0 = 1$. Since $f(n) = \Theta(n) = \Omega(n^{0+\epsilon})$, this is Case 3 (root dominates): $T(n) = \Theta(f(n)) = \Theta(n)$.

Compare this to merge sort’s $a = 2$ (both halves recursed into) giving $\Theta(n \log n)$: the single crucial difference is that quickselect discards one entire side after partitioning rather than recursing into both, which is what collapses the geometric sum of per-level work ($n + n/2 + n/4 + \dots = \Theta(n)$, a convergent geometric series per Chapter 3) instead of the n appearing at every one of $\Theta(\log n)$ levels.

KEY INSIGHT

This is a powerful, general lesson in algorithm design: **whether you recurse into one branch or many is often the single biggest lever on asymptotic complexity** — far more impactful than micro-optimizing the constant-factor work done per call. Binary search (one branch, Chapter 4) is $\Theta(\log n)$; merge sort (two branches) is $\Theta(n \log n)$; a naive recursive Fibonacci (two branches, overlapping subproblems, no memoization) is $\Theta(2^n)$ — a preview of why Chapter 20 exists.

19.4 Strassen's Algorithm: Reducing the Branching Factor

STEP-BY-STEP DERIVATION Why Reducing a from 8 to 7 Matters

Naive recursive matrix multiplication of two $n \times n$ matrices splits each into four $(n/2) \times (n/2)$ blocks and computes 8 block-multiplications (recall Chapter 4's worked example): $T(n) = 8T(n/2) + \Theta(n^2)$, which is Case 1 of the Master theorem, giving $\Theta(n^{\log_2 8}) = \Theta(n^3)$ — no better than the straightforward triple-loop algorithm.

Strassen's insight was purely algebraic: through a clever combination of additions and subtractions, the same 2×2 block multiplication can be computed using only 7 multiplications instead of 8 (at the cost of more additions, which are cheaper). This changes the recurrence to $T(n) = 7T(n/2) + \Theta(n^2)$, watershed $n^{\log_2 7} \approx n^{2.807}$, still Case 1, giving

$$T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$$

— asymptotically better than $\Theta(n^3)$, achieved purely by reducing the *branching factor* a in the recurrence, with no change to how the problem is divided (b) or combined ($f(n)$'s order).

KEY INSIGHT

Strassen's algorithm is the clearest possible illustration that the Master Theorem is not just an analysis tool but a *design compass*: once you see that complexity is $\Theta(n^{\log_b a})$ in the branching-dominated case, you know exactly

which knob to turn to improve it — reduce a , the number of subproblems, even if it means more work per level combining them, as long as the extra combining work stays below the new watershed.

TAKEAWAY: *Divide and conquer's complexity is governed by three numbers you choose at design time — the branching factor a , the shrink factor b , and the combine cost $f(n)$ — and the Master Theorem tells you, before writing a single line of pseudocode, exactly which of these three choices dominates the final running time.*

CHAPTER 20

Dynamic Programming: Recurrences with Memory

20.1 The Two Mathematical Preconditions

DEFINITION Optimal Substructure and Overlapping Subproblems

Dynamic programming (DP) applies when a problem has:

1. **Optimal substructure:** an optimal solution to the problem contains, within it, optimal solutions to subproblems.
2. **Overlapping subproblems:** the naive recursive solution re-solves the *same* subproblem many times.

DP's entire mechanism is: write the recurrence exactly as you would for divide-and-conquer (Chapter 19), but **cache** each subproblem's answer the first time it's computed, so subsequent requests are $\Theta(1)$ lookups instead of re-computation.

20.2 Diagnosing Overlap: A Counting Argument

STEP-BY-STEP DERIVATION Why Naive Fibonacci Is $\Theta(2^n)$

The recurrence $F(n) = F(n-1) + F(n-2)$, computed by naive recursion with no caching, has a recursion tree where each call branches into 2 children (matching the Master Theorem's $a = 2$ shape from Chapter 4, but with $b = 1$ -ish shrinkage, i.e., outside the standard Master Theorem's applicability, since subproblem sizes shrink by a constant amount, not a constant factor). Because the tree has depth $\Theta(n)$ and branches by (approximately) 2 at every node,

$$T(n) = T(n-1) + T(n-2) + \Theta(1) \implies T(n) = \Theta(\phi^n) \subseteq \Theta(2^n)$$

using the same Fibonacci-growth fact established in Chapter 12's AVL analysis. Yet there are only $n + 1$ *distinct* subproblems, $F(0)$ through $F(n)$! The exponential blowup is caused entirely by re-solving the same handful of subproblems exponentially many times — $F(n - 2)$ alone is computed twice, $F(n - 3)$ three times, and so on, a redundancy that grows combinatorially with n .

KEY INSIGHT

This gap — $n + 1$ distinct subproblems versus $\Theta(2^n)$ total calls — is the single diagnostic question for spotting a DP opportunity: **count the number of distinct subproblems (typically a polynomial function of the input parameters), and compare it to the exponential blowup of the naive recursion tree.** If distinct subproblems are polynomially few but the naive recursion revisits them exponentially often, caching converts the algorithm's complexity from exponential to (number of subproblems) $\times$ (work per subproblem).

20.3 Two Implementations, One Recurrence

DEFINITION Memoization vs. Tabulation

Memoization (top-down): keep the natural recursive structure, but check a cache before recursing, and store the result after computing it. **Tabulation** (bottom-up): identify a valid evaluation order for all subproblems (smallest to largest) and fill a table iteratively, with no recursion at all.

STEP-BY-STEP DERIVATION Why Both Give the Same $\Theta(n)$ for Fibonacci

With caching (either style), each of the $n + 1$ distinct subproblems $F(0), \dots, F(n)$ is computed exactly once, doing $\Theta(1)$ work per subproblem (one addition, one cache read for each of two smaller subproblems). Total time:

$$T(n) = \Theta(n) \cdot \Theta(1) = \Theta(n)$$

— an exponential-to-linear improvement, achieved purely by eliminating redundant work, with *zero* change to the mathematical recurrence itself.

20.4 Worked Example: 0/1 Knapsack

DEFINITION 0/1 Knapsack Recurrence

Given n items with weights w_i and values v_i , and a capacity W , define $K(i, c)$ as the best achievable value using only the first i items with total weight $\leq c$. The recurrence considers whether item i is included:

$$K(i, c) = \max \left(\underbrace{K(i-1, c)}_{\text{exclude item } i}, \underbrace{v_i + K(i-1, c - w_i)}_{\text{include item } i, \text{ if } w_i \leq c} \right), \quad K(0, c) = 0.$$

STEP-BY-STEP DERIVATION Complexity from Counting the Subproblem Grid

The subproblem space is a 2D grid indexed by $i \in \{0, \dots, n\}$ and $c \in \{0, \dots, W\}$: exactly $(n+1)(W+1)$ distinct subproblems, each computed in $\Theta(1)$ given the two smaller subproblems it depends on. Total:

$$T(n, W) = \Theta(nW).$$

KEY INSIGHT

Notice $T(n, W) = \Theta(nW)$ is polynomial in n and the *numeric value* W , not in the number of *bits* needed to represent W — this is called **pseudo-polynomial** time. If W is astronomically large (e.g., $W = 2^{100}$), this “polynomial” bound is actually useless in practice, since nW would be enormous. Recognizing the difference between true polynomial time and pseudo-polynomial time is itself a mathematical skill: always ask whether your DP’s subproblem count is polynomial in the *input size* (number of bits/items) or merely in the *numeric values* appearing in the input, since these can be exponentially different quantities.

20.5 Reconstructing the Solution, Not Just the Value

KEY INSIGHT

The recurrence above computes the *optimal value*, but recovering the actual set of chosen items requires tracing back through the filled table: at cell (i, c) , if $K(i, c) = K(i - 1, c)$, item i was excluded; otherwise it was included, and you continue tracing from $(i - 1, c - w_i)$. This backtracking-through-the-table costs only $O(n)$ extra (one step back per item), an important reminder that a DP table gives you both the optimal *value* and, via this cheap traceback, a full certificate of *which choices* achieved it — the recurrence’s structure doubles as a proof of optimal substructure for the reconstructed solution.

20.6 DP on a Different Axis: Longest Common Subsequence

DEFINITION LCS Recurrence

For strings X (length p) and Y (length q), define $L(i, j)$ as the length of the longest common subsequence of X ’s first i characters and Y ’s first j characters:

$$L(i, j) = \begin{cases} L(i - 1, j - 1) + 1 & \text{if } X_i = Y_j \\ \max(L(i - 1, j), L(i, j - 1)) & \text{if } X_i \neq Y_j \end{cases}, \quad L(0, \cdot) = L(\cdot, 0) = 0.$$

WORKED EXAMPLE Filling a Small LCS Table

For $X = \text{“ABC”}$, $Y = \text{“AC”}$: the grid computes $L(1, 1) = 1$ (A matches A), $L(2, 1) = 1$ (B vs A mismatch, carry max from left/above), $L(3, 1) = 1$, $L(3, 2) = 2$ (C matches C, plus the earlier $L(2, 1) = 1$). Final answer $L(3, 2) = 2$, corresponding to the subsequence “AC.” The grid has $(p + 1)(q + 1) = 4 \times 3 = 12$ cells, each $\Theta(1)$ work: $\Theta(pq)$ total — the same 2D-grid counting argument as knapsack, applied to a completely different problem domain, underscoring that it is the *shape* of the recurrence’s dependency structure, not the problem’s surface description, that determines DP complexity.

TAKEAWAY: *Dynamic programming is divide-and-conquer's recurrence (Chapter 19) plus one extra idea — cache every subproblem's answer — and the entire discipline of applying it reduces to two counting questions: how many distinct subproblems are there (this becomes your table's size), and how much work does each cell need given its dependencies (this becomes your per-cell cost)?*

CHAPTER 21

Greedy Algorithms: When Local Optimality Suffices

21.1 The Greedy Template, and Why It's Dangerous

DEFINITION Greedy Strategy

At each step, make the choice that looks best *right now* (by some local criterion), without reconsidering it later, and never backtrack. This is computationally cheap — typically $\Theta(1)$ or $\Theta(\log n)$ decision cost per step — but is only *correct* for problems with special mathematical structure.

KEY INSIGHT

Greedy algorithms and DP (Chapter 20) both rely on optimal substructure, but they part ways on overlapping subproblems: DP handles problems where the locally best choice can be wrong (so it must consider multiple options and cache results), while greedy applies only when we can *prove* the locally best choice is never wrong. That proof is not optional — a greedy algorithm without a correctness proof is just a heuristic that might silently give wrong answers on some inputs. Chapter 18's MST cut property and Chapter 6's activity-selection exchange argument are the two template proofs; this chapter generalizes the pattern.

21.2 The Matroid Abstraction (Informally)

DEFINITION Matroid — Informal Definition

A matroid is a mathematical structure $(E, \mathcal{I})$ — a ground set E and a family $\mathcal{I}$ of “independent” subsets of E — satisfying two axioms: (1) any subset of an independent set is independent (**hereditary property**); (2) if $A, B \in \mathcal{I}$ with $|A| < |B|$, some element of $B \setminus A$ can be added to A while keeping it

independent (**exchange property**).

KEY INSIGHT

The deep theorem here (stated without full proof, as it belongs to a more advanced text, but essential to know exists) is: **the greedy algorithm is guaranteed to find an optimal solution whenever the problem's feasible solutions form a matroid**. Kruskal's algorithm (Chapter 18) is greedy-on-a-matroid in disguise: E is the edge set, and $\mathcal{I}$ is the family of acyclic edge subsets (forests) — the hereditary property holds because a subset of a forest is still a forest, and the exchange property holds because you can always augment a smaller forest with an edge from a larger one without creating a cycle. Recognizing that a new problem's structure forms (or resembles) a matroid is one of the most powerful “does greedy even apply here?” litmus tests available.

21.3 Worked Proof: Fractional Knapsack

DEFINITION Fractional Knapsack

Like 0/1 knapsack (Chapter 20), but items can be taken in *any fraction* $0 \leq x_i \leq 1$, maximizing $\sum x_i v_i$ subject to $\sum x_i w_i \leq W$.

STEP-BY-STEP DERIVATION Why “Sort by Value-per-Weight” Is Optimal — Exchange Argument

Greedy claim: sort items by v_i/w_i descending, and greedily take as much as possible of each in that order.

Proof: suppose an optimal solution x^* does not follow this order — i.e., some item i with higher ratio v_i/w_i is taken less fully ($x_i^* < 1$) than some item j with a lower ratio ($x_j^* > 0$). Consider swapping a small amount ε of weight from item j to item i : decrease x_j by ε/w_j , increase x_i by ε/w_i , keeping total weight unchanged. The value changes by

$$\Delta = \varepsilon \cdot \frac{v_i}{w_i} - \varepsilon \cdot \frac{v_j}{w_j} > 0$$

since $v_i/w_i > v_j/w_j$ by assumption. This strictly improves the solution, contradicting x^* 's optimality. Hence no optimal solution can disagree with the greedy order. ■

Contrast this with 0/1 knapsack, where the exact same greedy order can be arbitrarily bad (imagine one item with a slightly-better ratio but huge weight, blocking two smaller equal-total-value items) — the continuous divisibility of the fractional version is precisely what the exchange argument's "swap a small amount ϵ " step requires, and it is unavailable once items become indivisible.

STEP-BY-STEP DERIVATION Running Time

Sorting n items by ratio: $\Theta(n \log n)$. Greedily filling the knapsack: a single $\Theta(n)$ pass. Total: $\Theta(n \log n)$, dominated by the sort — versus 0/1 knapsack's $\Theta(nW)$ pseudo-polynomial DP (Chapter 20), illustrating how a small change in problem structure (divisibility) can shift the right paradigm entirely, from DP to a much faster greedy solution.

21.4 Huffman Coding: Greedy Merging

DEFINITION Huffman's Strategy

Given character frequencies, repeatedly merge the two *least frequent* remaining nodes into a new node (frequency = sum of the two), until one tree remains. The resulting tree defines a variable-length binary code minimizing expected code length.

KEY INSIGHT

The correctness proof (omitted in detail here, but following the same shape as the others in this chapter) shows that the two least-frequent symbols can always be made siblings at the deepest level of *some* optimal tree — another exchange argument, this time on tree structure rather than a linear order. Using a min-heap (Chapter 13) to always extract the two smallest frequencies costs $O(\log n)$ per merge, over $n - 1$ merges: $\Theta(n \log n)$ total, another example of a heap turning an abstract greedy strategy into an efficient concrete

algorithm.

21.5 When Greedy Fails: A Cautionary Counterexample

WORKED EXAMPLE Greedy Fails on 0/1 Knapsack

Capacity $W = 10$. Item A: weight 6, value 12 (ratio 2). Item B: weight 5, value 9 (ratio 1.8). Item C: weight 5, value 9 (ratio 1.8). Greedy-by-ratio picks A first (weight 6, value 12), leaving capacity 4 — insufficient for B or C, final value 12. But taking B and C together uses weight 10 exactly for value 18 — strictly better. The exchange argument from the fractional case breaks down here specifically because items cannot be partially split to equalize the ε trade — a direct illustration of why the fractional proof’s key step is unavailable in the 0/1 setting, and why 0/1 knapsack genuinely needs DP’s ability to consider both “include” and “exclude” at every step instead of committing greedily.

TAKEAWAY: *A greedy algorithm is only as trustworthy as its correctness proof — usually an exchange argument (Chapter 6) or matroid-structure argument — and the sharpest way to develop algorithmic judgment is to actively look for the counterexample that would break greedy before trusting it, exactly as the 0/1 knapsack example shows.*

CHAPTER 22

Backtracking and Branch and Bound: Pruning the Search Tree

22.1 The State Space Tree

DEFINITION State Space Tree

Any problem solved by trying choices and undoing them on failure can be modeled as a tree: the root is the empty (partial) solution, each edge represents making one more choice, and leaves represent complete candidate solutions. Backtracking is a DFS (Chapter 16) over this tree that **prunes** — abandons — any branch that provably cannot lead to a valid or improving solution.

STEP-BY-STEP DERIVATION Why Brute Force Is Exponential — Counting the Tree

Consider the n -queens problem: place n queens on an $n \times n$ board so none attacks another. A brute-force state space tree that tries every possible placement of each queen in any of n rows across n columns has branching factor n at each of n levels, giving n^n leaves — for $n = 8$, that's $8^8 \approx 16.7$ million; for $n = 20$, astronomically larger. This matches the growth-rate hierarchy from Chapter 2 (n^n sits above even 2^n and $n!$), which is exactly why unpruned brute force is unusable beyond small n .

22.2 Pruning as Reducing the Effective Branching Factor

KEY INSIGHT

Backtracking's entire value proposition is mathematical: it does not change the *worst-case* size of the state space tree, but it can drastically reduce the *actual number of nodes visited* by cutting off branches early, the moment a partial solution is detected to violate a constraint — no need to complete a doomed branch just to discover it fails at the leaf. For n -queens, placing queens one row at a time and immediately checking column/diagonal conflicts reduces the effective branching factor at each level from n down to (at most) n but typically far fewer valid columns, since most placements are eliminated within the first few rows.

WORKED EXAMPLE Quantifying the Pruning Effect for 8-Queens

Brute force explores up to $8^8 \approx 16,777,216$ leaf placements. With row-by-row placement and immediate column/diagonal conflict checking, the actual number of nodes visited by backtracking is on the order of a few thousand — several orders of magnitude fewer. The *shape* of the state space tree hasn't changed (it's still bounded by 8^8 in the worst theoretical case), but pruning ensures we never materialize the vast majority of it, because most branches die within the first 2–3 levels.

22.3 Branch and Bound: Pruning by a Numerical Bound

DEFINITION Branch and Bound

An extension of backtracking for optimization problems (not just feasibility): maintain the best solution found so far (the **incumbent**), and for every partial solution, compute an optimistic **bound** on the best value reachable from that branch. If the bound is no better than the incumbent, prune the entire branch — it cannot possibly contain the optimum.

STEP-BY-STEP DERIVATION Why a Tighter Bound Prunes More — An Inequality Argument

Let $\text{OPT}(\text{node})$ be the true best achievable value from a partial solution, and $\text{bound}(\text{node}) \geq \text{OPT}(\text{node})$ be a computable over-estimate (for maximization). A branch is safely pruned whenever $\text{bound}(\text{node}) \leq \text{value}(\text{incumbent})$, since this guarantees $\text{OPT}(\text{node}) \leq \text{bound}(\text{node}) \leq \text{value}(\text{incumbent})$ — the branch cannot beat what we already have. The tighter (numerically smaller, for maximization) a valid bound is, the more branches satisfy this pruning inequality, and the fewer nodes get explored. This is precisely why, for example, the linear-programming relaxation bound used in branch-and-bound for integer programming (solving the fractional knapsack version from Chapter 21 as a bound for the 0/1 version from Chapter 20) is so valuable: it is provably at least as good as the true integer optimum, yet cheap to compute, giving a mathematically sound and often very tight pruning criterion.

KEY INSIGHT

This connects directly back to Chapter 21's fractional-vs-0/1 knapsack example: the fractional (greedy, $\Theta(n \log n)$) solution's value is always $\geq$ the true 0/1 optimum for the same instance, since 0/1 solutions are a strict subset of fractional ones. This makes it a perfect branch-and-bound bound for solving 0/1 knapsack exactly — using a fast, provably-correct relaxation to prune a search that would otherwise be exponential, blending greedy reasoning (Chapter 21) and exhaustive search into one hybrid strategy.

22.4 Backtracking's Relationship to Recursion and Induction

KEY INSIGHT

Just as recursion mirrors induction (Chapter 6) and divide-and-conquer's correctness follows the same pattern, backtracking's correctness is essentially a completeness argument: as long as every prune step only discards branches that *provably* cannot contain a valid/optimal solution (verified precisely by the bound or constraint-check argument), the search remains ex-

haustive over everything that *could* matter, and is therefore guaranteed to find the true answer if one exists — pruning trades away redundant exploration, never correctness, provided the pruning condition is a genuine logical implication rather than a heuristic guess.

TAKEAWAY: *Backtracking and branch-and-bound do not change the theoretical worst-case size of a state-space tree — they change how much of it you actually have to visit, and the entire craft lies in finding cheap, valid, and tight numerical bounds or constraint checks that let you prove a branch is hopeless before fully exploring it.*

Part VII

Advanced Analysis

CHAPTER 23

Amortized Analysis: Averaging Over Time, Rigorously

23.1 Why Worst-Case-Per-Operation Can Be Too Pessimistic

We have already met amortized reasoning informally twice: dynamic array doubling (Chapter 7) and hash table resizing (Chapter 15). This chapter makes the technique rigorous with three formal methods, so you can apply it confidently to new structures rather than by analogy.

DEFINITION Amortized Cost

The **amortized cost** of an operation is an upper bound on the *average* cost per operation across any sequence of n operations, i.e., $\frac{1}{n} \sum \text{actual cost}_i \leq \text{amortized cost}$, even though individual operations may vary wildly above or below that amortized figure.

23.2 Method 1: Aggregate Analysis

STEP-BY-STEP DERIVATION Dynamic Array Doubling, Redone via Aggregate Analysis

Sum the *total* cost of n insertions directly, then divide by n . As derived in Chapter 7, total copying cost across all resizes is bounded by the geometric series $1 + 2 + 4 + \dots < 2n$, plus n for the n ordinary $\Theta(1)$ insertions themselves. Total cost $< 3n$, so

$$\text{amortized cost per insertion} = \frac{3n}{n} = \Theta(1).$$

This is the simplest amortized method: just sum everything and divide.

23.3 Method 2: The Accounting Method

DEFINITION Accounting Method

Assign each operation an **amortized charge** (possibly different from its actual cost). Operations that are charged more than their actual cost build up **credit**, stored on the data structure; operations that cost more than their charge spend down previously accumulated credit. The method is valid if credit never goes negative.

STEP-BY-STEP DERIVATION Dynamic Array via Accounting

Charge each insertion a fee of 3 (amortized cost), even though an ordinary insertion only actually costs 1. The extra 2 becomes credit stored on the newly inserted element. **Claim:** this is always enough to pay for the next resize. When the array (holding k elements, capacity k) fills up and must double, the resize must copy all k elements. Of those k elements, exactly the most recent $k/2$ (inserted since the last resize) are still carrying their unspent credit of 2 each — a total of k available credit, exactly enough to pay the k -element copying cost. Since credit never runs negative, charging 3 per insertion is a valid amortized bound, matching the aggregate method's result via a different bookkeeping device.

KEY INSIGHT

The accounting method's real value shows up on structures too irregular for a clean aggregate sum. A classic example: a binary counter where increment flips a suffix of bits — charge 2 per increment (one to flip a $0 \rightarrow 1$, one saved as credit on that bit); when a later increment must flip a run of 1s back to 0, each such flip is paid for by the credit stored when that bit was set to 1. This gives $\Theta(1)$ amortized time per increment, even though a single increment can, in the worst case, flip $\Theta(\log n)$ bits.

23.4 Method 3: The Potential Method

DEFINITION Potential Method

Define a **potential function** Φ mapping the data structure's state to a non-negative real number, with $\Phi(\text{initial state}) = 0$ (or a known baseline). The amortized cost of an operation is defined as

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

where c_i is the actual cost and D_i is the structure's state after operation i . Summing telescopes:

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0).$$

Since $\Phi(D_n) \geq 0$ and $\Phi(D_0) = 0$, this proves $\sum c_i \leq \sum \hat{c}_i$ — the amortized costs upper-bound the true total cost, provided each individual $\hat{c}_i$ can be bounded.

STEP-BY-STEP DERIVATION Dynamic Array via the Potential Method

Define $\Phi(D) = 2k - \text{capacity}$, where k is the current number of elements (i.e., twice the number of elements inserted since the last resize, matching the “credit per element” intuition from the accounting method, but now expressed as a single global function of the state). For an ordinary insertion (no resize), $c_i = 1$ and capacity is unchanged while k increases by 1, so $\Delta\Phi = 2$, giving $\hat{c}_i = 1 + 2 = 3$. For a resize-triggering insertion, $c_i = k + 1$ (copy all k old elements, plus insert the new one), and capacity doubles from k to $2k$ while the element count becomes $k + 1$; Φ before the resize was $2k - k = k$, and after is $2(k + 1) - 2k = 2$, so $\Delta\Phi = 2 - k$, giving $\hat{c}_i = (k + 1) + (2 - k) = 3$. Every operation, resize or not, has amortized cost exactly $3 = \Theta(1)$ — matching both earlier methods, now derived from a single formula rather than an ad hoc credit-tracking argument.

KEY INSIGHT

The three methods are not competing theories — they are three lenses on the same underlying truth, and the potential method is the most powerful because Φ can be engineered to make almost any amortized argument mechanical: choose Φ to measure “how much expensive work has been deferred and is now stored up in the structure’s current state,” and the telescoping sum does the rest of the proof automatically. When faced with a new structure with occasional expensive operations, the design question is always: what quantity, if it’s currently large, predicts that the structure is “due” for an expensive operation soon? That quantity, suitably scaled, is your Φ .

TAKEAWAY: *Amortized analysis proves that a sequence of operations is cheap on average, even when individual operations are expensive, using one of three equivalent formal tools — aggregate summing, accounting credit, or a potential function — and the same doubling/geometric-growth idea that made dynamic arrays and hash tables $\Theta(1)$ amortized reappears as the template for nearly every amortized argument you will encounter.*

CHAPTER 24

Randomized Algorithms: Probability as a Design Tool

24.1 Two Flavors of Randomization

DEFINITION Las Vegas vs. Monte Carlo

A **Las Vegas** algorithm always produces a correct answer, but its *running time* is a random variable (e.g., randomized quicksort, Chapter 5). A **Monte Carlo** algorithm has a fixed running time, but its *correctness* is probabilistic — it may output a wrong answer with some bounded probability.

24.2 Revisiting Randomized Quicksort’s Guarantee

KEY INSIGHT

Chapter 5 proved $\mathbb{E}[\text{comparisons}] = \Theta(n \log n)$ for randomized quicksort using linearity of expectation. It is worth stating precisely what this guarantee does and does not promise: it does *not* claim every run is fast — an adversary (or sheer bad luck) can still produce a $\Theta(n^2)$ run, since the worst-case input for *any fixed pivot rule* still exists in principle. What randomization achieves is that *no adversary, even one who sees your code, can reliably construct a bad input in advance*, because the pivot choice depends on random numbers unknown until runtime — the exact same defense mechanism as universal hashing in Chapter 15.

24.3 Randomized Algorithms as a Tool for Simplicity: Quickselect

We derived in Chapter 19 that randomized quickselect runs in expected $\Theta(n)$ time via the Master Theorem’s Case 3, using the same “expected balanced split”

argument as quicksort. It is worth re-deriving this via direct probability, since it illustrates a second core technique: bounding the probability of a “good” split directly.

STEP-BY-STEP DERIVATION Probability of a Reasonably Balanced Pivot

Call a pivot “good” if it lands within the middle 50% of the sorted order (ranks between $n/4$ and $3n/4$). Since a uniformly random pivot lands in this range with probability exactly $1/2$ (half of all ranks fall in the middle half, by definition), the expected number of pivot choices needed until a good one is found is $\mathbb{E}[\text{trials}] = 1/\Pr[\text{good}] = 2$ (a direct application of the expectation of a geometric distribution). Each good pivot shrinks the problem by at least a factor of $3/4$. So the expected recursion depth to reach a trivial size is bounded, and the expected total work is

$$\mathbb{E}[T(n)] \leq \mathbb{E}[T(3n/4)] + \Theta(n)$$

which by the Master Theorem (Case 3, since $f(n) = \Theta(n)$ dominates $n^{\log_{4/3} 1} = n^0$) gives $\mathbb{E}[T(n)] = \Theta(n)$ — confirming the earlier result via an explicit “probability a step is good” argument rather than an average-case split-size argument. This “define good/bad, bound $\Pr[\text{good}]$, bound work-per-good-step” template generalizes to analyzing many other randomized algorithms.

24.4 Monte Carlo Example: Randomized Primality-Style Verification

DEFINITION Freivalds’ Algorithm — Verifying Matrix Multiplication

Given $n \times n$ matrices A, B, C , verifying $AB = C$ directly costs $\Theta(n^3)$ via the naive algorithm (Chapter 19). Freivalds’ algorithm instead picks a random 0/1 vector r of length n and checks only whether $A(Br) = Cr$ — three matrix-vector products, each $\Theta(n^2)$, so $\Theta(n^2)$ total: asymptotically faster than recomputing the full product.

STEP-BY-STEP DERIVATION Bounding the Error Probability

If $AB = C$ truly, the check always passes ($A(Br) = (AB)r = Cr$ exactly) — no false negatives. If $AB \neq C$, let $D = AB - C \neq 0$; one can show that for a random 0/1 vector r , $\Pr[Dr = 0] \leq 1/2$ (a consequence of D having at least one non-zero row, and a pigeonhole-style counting argument over the 2^n possible vectors r , half of which fail to zero out any fixed non-zero linear form). So a single run has at most a 50% chance of a false positive (wrongly reporting $AB = C$). Running the check k independent times and requiring all k to pass reduces the false-positive probability to $\leq (1/2)^k$ — with $k = 20$, error probability below one in a million, at a total cost of $\Theta(kn^2)$, still far cheaper than the exact $\Theta(n^3)$ (or even Strassen's $\Theta(n^{2.807})$, Chapter 19) recomputation for large n .

KEY INSIGHT

This “repeat k times, multiply the error probability” technique is the standard way to convert a Monte Carlo algorithm with a merely-adequate ($\leq 1/2$) error bound into one with an overwhelmingly small error bound, at only a linear (not exponential) cost increase in k — because independent trials' failure probabilities *multiply*, shrinking geometrically fast (echoing the geometric series machinery from Chapter 3 once more, here operating on probabilities instead of costs).

24.5 The General Lesson: Randomization Buys Simplicity or Speed

KEY INSIGHT

Across this book, randomization has appeared to solve two distinct kinds of problems: (1) defeating an adversary who could otherwise force worst-case behavior on a deterministic algorithm (quicksort's pivot, hash table's universal family), and (2) achieving an asymptotically faster algorithm than any known deterministic one for the same guarantee (Freivalds' $\Theta(n^2)$ verification versus $\Theta(n^{2.807})$ deterministic recomputation). Recognizing which of these two benefits you actually need is itself a design decision: if you only need protection against adversarial inputs, a Las Vegas approach preserving

exact correctness (like quicksort) is usually preferable; if you can tolerate a tunably tiny error probability in exchange for real asymptotic speedup, a Monte Carlo approach becomes worth considering.

TAKEAWAY: *Randomized algorithms use the probability toolkit from Chapter 5 — expectation, linearity of expectation, and probability bounds — as a first-class design tool, either to make worst-case inputs statistically unconstructable (Las Vegas) or to trade a controllably small, geometrically-shrinkable error probability for real asymptotic speed (Monte Carlo).*

CHAPTER 25

P, NP, and the Limits of Efficient Computation

25.1 Defining the Classes

DEFINITION P and NP

P is the class of decision problems (yes/no questions) solvable by a deterministic algorithm in time polynomial in the input size n , i.e., $O(n^k)$ for some constant k . **NP** is the class of decision problems for which a proposed “yes” answer, together with a **certificate** (a witness/proof), can be *verified* in polynomial time — regardless of how hard the certificate was to find in the first place.

KEY INSIGHT

Every problem in **P** is trivially in **NP** (if you can solve it in polynomial time, you certainly can verify a proposed answer in polynomial time — just solve it yourself and compare). The single biggest open question in computer science, $\mathbf{P} \stackrel{?}{=} \mathbf{NP}$, asks whether the reverse also holds: can every efficiently *verifiable* problem also be efficiently *solved*? All evidence accumulated over decades of study suggests no, but no proof exists.

WORKED EXAMPLE Sudoku as an NP Problem

Given a candidate filled-in Sudoku grid, checking that every row, column, and box contains each digit exactly once is a simple $O(n^2)$ scan — fast verification. But *finding* a valid filling from a partially-filled puzzle, in the worst case (generalized to $n \times n$ grids), is believed to require time exponential in n in the worst case. This gap — cheap to check, seemingly expensive to solve — is the defining texture of every NP-complete problem.

25.2 Reductions: The Mathematics of “At Least As Hard”

DEFINITION Polynomial-Time Reduction

Problem A **reduces** to problem B (written $A \leq_p B$) if there is a polynomial-time algorithm transforming any instance of A into an instance of B such that: answer to A 's instance is yes $\iff$ answer to the transformed B instance is yes.

KEY INSIGHT

A reduction $A \leq_p B$ means “ B is at least as hard as A ” — if we ever found a fast algorithm for B , we would immediately get a fast algorithm for A too, just by running the reduction first. This gives reductions their power as a proof technique: to show a *new* problem X is hard, we don't need to reason about X from scratch — we only need to find some *already known* hard problem A and construct a reduction $A \leq_p X$. This is the exact same “prove a smaller, related fact and build on it” spirit as induction (Chapter 6), applied to relative hardness instead of absolute truth.

DEFINITION NP-Complete

A problem is **NP-complete** if (1) it is in **NP**, and (2) every other problem in **NP** reduces to it in polynomial time. An NP-complete problem is therefore, in a precise sense, “as hard as any problem in **NP** can be” — solving any single NP-complete problem in polynomial time would imply $P = NP$, i.e., would solve *every* problem in **NP** efficiently.

25.3 A Worked Reduction: Vertex Cover from 3-SAT (Sketch)

WORKED EXAMPLE The Reduction Pattern, Illustrated

The classical proof that VERTEX COVER is NP-complete reduces from 3-SAT (given a Boolean formula in 3-CNF form, does some variable assignment make it true?), itself proved NP-complete first (via the Cook-Levin theorem, the historical “first” NP-complete problem, proved directly from the defi-

inition of NP rather than by reduction). The construction builds, for each clause and each variable, small constant-size “gadget” subgraphs whose vertex-cover structure exactly mirrors the true/false logic of a satisfying assignment — a satisfying assignment to the formula exists if and only if the constructed graph has a vertex cover of a specific computed size. Building this graph takes time polynomial in the formula’s size, satisfying the reduction’s efficiency requirement. Reasoning through such a construction in full is beyond this book’s scope, but the *shape* of the argument — encode logical structure as graph structure, gadget by gadget, then prove the equivalence in both directions — is the template every NP-completeness proof follows.

25.4 What to Do When You Suspect NP-Hardness

KEY INSIGHT

Recognizing that a problem you are trying to solve efficiently is (or strongly resembles) a known NP-complete problem is, itself, valuable design information — it tells you to stop searching for a polynomial-time *exact* algorithm and pivot to one of several standard fallback strategies: (1) exponential but heavily **pruned** search (Chapter 22’s backtracking and branch-and-bound), acceptable when instance sizes stay modest; (2) **approximation algorithms** that provably return a solution within some bounded factor of optimal in polynomial time (e.g., a simple 2-approximation for vertex cover exists, even though the exact problem is NP-complete); (3) restricting to special-case inputs where the problem becomes tractable (e.g., vertex cover is solvable in polynomial time on trees, via a DP nearly identical in spirit to Chapter 20’s techniques, since trees lack the general graph structure the hardness proof relies on); or (4) randomized/heuristic methods (Chapter 24) that work well in practice without a worst-case guarantee.

TAKEAWAY: *NP-completeness is not a statement that a problem is unsolvable — it is a precise, reduction-based mathematical certificate that a problem is at least as hard as every other problem efficiently verifiable at all, and recognizing this classification is what tells you to stop searching for an exact polynomial algorithm and reach instead for pruning, approximation, restricted special cases, or randomization.*

Part VIII

Synthesis

CHAPTER 26

A Framework for Designing Efficient Algorithms

You now have every mathematical tool used in this book. This closing chapter converts them into an active checklist — a sequence of questions to ask when facing a brand-new problem, each one pointing back to the chapter that equips you to answer it.

26.1 Step 1: Characterize the Problem Mathematically

- **What is the input size parameter(s)?** Not just “ n ” — knapsack (Chapter 20) taught us to separately track n (item count) and W (capacity), since conflating them hides pseudo-polynomial blowups.
- **What is the state space?** How many distinct configurations/subproblems exist? (Chapter 20’s core diagnostic question for spotting DP opportunities.)
- **Is there a natural recursive decomposition?** If solving the whole requires solving smaller versions of itself, you are in divide-and-conquer/DP/backtracking territory (Chapters 19, 20, 22).

26.2 Step 2: Choose an Invariant

Recall Chapter 1’s framing: every data structure is an answer to “what invariant do I maintain, and what does maintaining it cost?”

- Need $\Theta(1)$ positional access? An array-style closed-form address formula (Chapter 7) or a heap’s index arithmetic (Chapter 13).
- Need fast search over an unbounded key space? A balanced-tree invariant (Chapter 12) for ordered operations, or a hash invariant (Chapter 15) for unordered $\Theta(1)$ -expected lookup.
- Need prefix-based queries? A trie’s shared-path invariant (Chapter 14).

- Need cheap insertion at known positions but don't need indexed access? A linked structure (Chapter 8).

26.3 Step 3: If Recursive, Write the Recurrence and Classify It

- Does the recursion branch into *multiple independent* subproblems whose results must be combined? Try the Master Theorem (Chapter 4) — compute $n^{\log_b a}$ and compare it against the combine cost $f(n)$.
- Do the subproblems *overlap* (the same subproblem needed more than once)? Switch from plain recursion to memoization/tabulation — you are doing DP (Chapter 20), not divide-and-conquer.
- Does exploring one branch let you *discard* the others entirely, with proof? Look for an exchange argument (Chapter 6) certifying a greedy choice (Chapter 21), or a cut-property-style structural argument (Chapter 18).

26.4 Step 4: Decide Worst-Case, Average-Case, or Amortized Is the Right Lens

- Do you need an ironclad guarantee on *every* input, including adversarial ones? Analyze worst case, and consider randomization (Chapter 24) specifically to neutralize adversarial inputs, or a strict balance invariant (Chapter 12) instead of a hopeful one.
- Is occasional expensive behavior acceptable as long as it's rare? Reach for amortized analysis (Chapter 23) — check whether a doubling/geometric-growth argument applies.
- Are you willing to accept a probabilistic (expected-value) guarantee in exchange for simplicity or speed? Use the probability toolkit of Chapter 5 directly — define the random variable you care about, and compute $\mathbb{E}[\cdot]$ via linearity of expectation whenever the quantity can be decomposed into simpler indicator pieces.

26.5 Step 5: Check for a Lower Bound Before Chasing an Impossible Speedup

- Is this a comparison-based sorting/selection problem? Recall the $\Omega(n \log n)$ counting-argument lower bound from Chapter 5 — don't hunt for a comparison-based sort faster than $n \log n$, because none exists.
- Does the problem resemble a known NP-complete problem? Look for a reduction (Chapter 25) before investing in an exact polynomial algorithm; pivot to pruning, approximation, or restricted-input strategies instead.
- Is there a structural lower bound specific to the data structure (e.g., $\Omega(\log n)$ height for any binary tree, Chapter 10)? Use it to recognize when an existing algorithm is already asymptotically optimal, so effort is better spent on constant-factor engineering rather than a fundamentally faster asymptotic approach.

26.6 A Worked Walkthrough of the Framework

WORKED EXAMPLE Applying the Checklist to a New Problem

Suppose you must design a system that repeatedly answers “what are the k most frequent search queries typed in the last hour?” as new queries stream in continuously.

Step 1: input parameter is the stream length n (unbounded, continuous) and k (small, fixed). State space: a frequency count per distinct query — track distinct queries, not raw stream length.

Step 2: we need fast increment-by-one-on-arrival ($\Theta(1)$ -ish) and fast “top- k ” retrieval. A hash map (Chapter 15) gives $\Theta(1)$ expected increment by query string. A small heap of size k (Chapter 13) gives $O(\log k)$ maintenance of the current top- k as counts change.

Step 3: no interesting recursive decomposition here — this is fundamentally an online data-structure-maintenance problem, not a divide-and-conquer or DP one.

Step 4: worst-case per-update cost is what matters here (a real-time system), so we favor the hash map's $O(1)$ expected (Chapter 15) combined with the heap's $O(\log k)$ worst-case (Chapter 13) — noting the small, con-

trolled risk that hashing's expected bound could be defeated by adversarial query strings, mitigated by universal hashing (Chapter 15) if the system is public-facing.

Step 5: no known matching lower bound forces anything slower — $O(\log k)$ per update, with k small and fixed, is about as good as any comparison-based structure can offer for exact top- k maintenance (recall Chapter 5's counting-based lower bounds apply to comparison-based selection generally).

The result: a hash-map-plus-bounded-heap design, arrived at entirely by walking the checklist rather than by pattern-matching to a memorized “top- k streaming” recipe.

TAKEAWAY: *Algorithm design is not pattern memorization — it is the repeated, disciplined application of five questions: what is the state space, what invariant should I maintain, does a recurrence classify cleanly under the Master Theorem or DP, which cost lens (worst/average/amortized) fits my requirements, and does a known lower bound tell me when to stop searching for a faster exact algorithm.*

CHAPTER 27

Complexity Cheat Sheet

This closing chapter is a compact reference, not new material — every entry below was derived, not asserted, somewhere earlier in this book. Use it for review, and use the chapter references to re-derive any entry you don't immediately trust from memory; re-deriving is always better than memorizing.

27.1 Data Structure Operations

Structure	Access	Search	Insert	Delete
Array (Ch. 7)	$\Theta(1)$	$O(n)$	$O(n)$	$O(n)$
Dynamic Array (Ch. 7)	$\Theta(1)$	$O(n)$	$\Theta(1)$ amort.	$O(n)$
Linked List (Ch. 8)	$O(n)$	$O(n)$	$\Theta(1)^\dagger$	$\Theta(1)^\dagger$
Stack / Queue (Ch. 9)	—	—	$\Theta(1)$	$\Theta(1)$
BST, average (Ch. 11)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
BST, worst case (Ch. 11)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
AVL / Red-Black (Ch. 12)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Heap (Ch. 13)	—	$O(n)$	$O(\log n)$	$O(\log n)^\ddagger$
Hash Table (Ch. 15)	—	$\Theta(1)$ exp.	$\Theta(1)$ exp.	$\Theta(1)$ exp.
Trie (Ch. 14)	—	$\Theta(L)$	$\Theta(L)$	$\Theta(L)$

† given a pointer to the location; finding that location is $O(n)$. ‡ extract-min/max specifically. L = key length.

27.2 Sorting Algorithms

Algorithm	Best	Average	Worst	Space
Insertion Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$	$O(1)$
Merge Sort (Ch. 4)	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$	$O(n)$
Quicksort, random pivot (Ch. 5)	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort (Ch. 13)	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$	$O(1)$

KEY INSIGHT

Every *comparison-based* algorithm above is bounded below by $\Omega(n \log n)$ in the worst case — a direct consequence of the $\log_2(n!)$ counting argument from Chapter 5, not a coincidence of these five particular algorithms.

27.3 Graph Algorithms

Algorithm	Time	Notes (Ch. 16–18)
BFS / DFS	$\Theta(n + m)$	Unweighted shortest path (BFS); cycle detect, topo sort (DFS)
Dijkstra (heap)	$O((n + m) \log n)$	Requires non-negative weights
Bellman-Ford	$O(nm)$	Handles negative weights; detects negative cycles
Floyd-Warshall	$O(n^3)$	All-pairs; simple DP recurrence
Kruskal's MST	$O(m \log n)$	Sort + union-find
Prim's MST (heap)	$O((n + m) \log n)$	Structurally mirrors Dijkstra

27.4 The Master Theorem, One More Time

For $T(n) = aT(n/b) + f(n)$, compare $f(n)$ to $n^{\log_b a}$ (Chapter 4):

Condition	Result	Example (Ch. 4, 13, 19)
$f(n) = O(n^{\log_b a - \epsilon})$	$T(n) = \Theta(n^{\log_b a})$	Naive 8-way matrix mult.: $\Theta(n^3)$
$f(n) = \Theta(n^{\log_b a})$	$T(n) = \Theta(n^{\log_b a} \log n)$	Merge sort: $\Theta(n \log n)$
$f(n) = \Omega(n^{\log_b a + \epsilon})$	$T(n) = \Theta(f(n))$	Randomized quickselect: $\Theta(n)$

27.5 Growth Rate Hierarchy (Chapter 2)

$$\begin{aligned} \Theta(1) &< \Theta(\log n) < \Theta(\sqrt{n}) < \Theta(n) < \Theta(n \log n) \\ &< \Theta(n^2) < \Theta(n^3) < \Theta(2^n) < \Theta(n!) \end{aligned}$$

27.6 The Proof Techniques Toolbox (Chapter 6)

Technique	Use when proving...
Induction	A recursive algorithm is correct
Loop invariant	An iterative algorithm is correct
Contradiction	A property can never be violated
Exchange argument	A greedy choice is safe / optimal
Reduction	A problem is at least as hard as another (Ch. 25)

27.7 Closing Words

Every number in every table above is a compressed proof — a summation evaluated (Chapter 3), a recurrence solved (Chapter 4), a counting argument completed (Chapter 5), or an invariant traced through to its cost (every structural chapter in this book). The goal was never for you to memorize these tables; it was for you to be able to rebuild any row of them, from first principles, on a blank page. That is the actual, durable skill: not knowing that quicksort is $\Theta(n \log n)$ on average, but being able to sit down, define the right indicator random variables, and show it yourself, the way Chapter 5 did. Carry the five-question framework from Chapter 26 into every new problem you meet, and let the mathematics — not memory — tell you what to build.